

TinyDB Query Engine and SQL Parser

A complete pipeline from raw SQL text to an executed result:

```
"SELECT name FROM employee WHERE salary > 50000"
  |
  v
SqlLexer      -- text -> tokens
  |
  v
SqlParser     -- tokens -> SqlStatement (an immutable AST)
  |
  v
QueryEngine   -- Level 1: execute the AST directly
               -- Level 2: hand it to a Planner instead
  |
  v
StorageEngine -- built in the companion Storage Engine guide
```

High-Level Architecture

Lexer → Parser → AST → Executors/Planner → Scan Pipeline

Two execution strategies are built in this guide, deliberately in sequence, not as alternatives to choose between arbitrarily:

```
Level 1 (Part 3): SqlStatement -> Executor -> StorageEngine directly
    —> Simple, correct, materializes full result sets in memory.

Level 2 (Part 4-5): SqlStatement -> Planner -> Plan -> Scan -> StorageEngine
    —> More moving parts, streams rows one at a time, and is the
        shape a Cost-Based Optimizer (Part 6) can eventually plug into.
```

Project Structure

```

sql-parser/
├── pom.xml
├── README.md
├── docs/
│   ├── grammar.md
│   ├── architecture.md
│   └── dialects.md
└── src/
    ├── main/java/com/sqlparser/
    │   ├── api/
    │   ├── source/
    │   ├── diagnostics/
    │   ├── lexer/
    │   ├── parser/
    │   ├── ast/
    │   ├── type/
    │   ├── semantic/
    │   ├── normalize/
    │   ├── plan/
    │   ├── optimizer/
    │   ├── physical/
    │   ├── executor/
    │   ├── dialect/
    │   ├── formatter/
    │   └── cli/
    └── test/java/com/sqlparser/
        ├── lexer/
        ├── parser/
        ├── semantic/
        ├── optimizer/
        └── formatter/

```

```

plan
├── LogicalPlan.java
├── LogicalScan.java
├── LogicalFilter.java
├── LogicalProject.java
├── LogicalJoin.java
├── LogicalAggregate.java
└── LogicalSort.java

```

```

optimizer
├── OptimizationRule.java
├── PlanOptimizer.java
├── PredicatePushdown.java
└── ProjectionPruning.java

```

```

physical
├── PhysicalPlan.java
├── PhysicalScan.java
├── HashJoin.java
├── NestedLoopJoin.java
└── SortOperator.java

```

```

executor
├── Executor.java
├── PhysicalOperator.java
└── Row.java

```

```

cli
└── SqlCli.java

```

```

lexer
├── SqlLexer.java
├── Token.java, TokenType.java
└── KeywordTable.java

ast
├── SqlStatement.java // sealed interface
├── SqlNode
├── SelectStatement, InsertStatement
├── UpdateStatement, DeleteStatement.java
├── CreateTableStatement, DropTableStatement
├── Expression.java
├── BinaryExpression.java
├── FunctionCallExpression.java
├── SubqueryExpression.java
├── Comparison.java, And.java, Or.java
├── Join.java
└── ...

```

```

api
├── SqlParser.java
├── ParseResult.java
└── ParserConfiguration.java

```

```

parser
├── SqlParserImpl.java
├── SelectParser.java
├── ExpressionParser.java
├── DmlParser.java
├── DdlParser.java
└── TokenStream.java

```

```

executor
├── StatementExecutor, SelectExecutor
├── InsertExecutor, UpdateExecutor
├── DeleteExecutor, CreateTableExecutor
├── DeleteExecutor, CreateTableExecutor
└── QueryEngine.java

```

```

source
├── SqlSource.java
└── SourceLocation.java

```

```

diagnostics
├── Diagnostic.java
└── DiagnosticReporter.java

```

```

type
├── SqlType.java
├── IntType.java
├── DecimalType.java
├── VarcharType.java
└── ...

```

```

semantic
├── SemanticAnalyzer.java
├── Scope.java
├── Catalog.java
├── TableMetadata.java
├── ColumnMetadata.java
└── FunctionRegistry.java

```

```

formatter
└── SqlFormatter.java

```

```

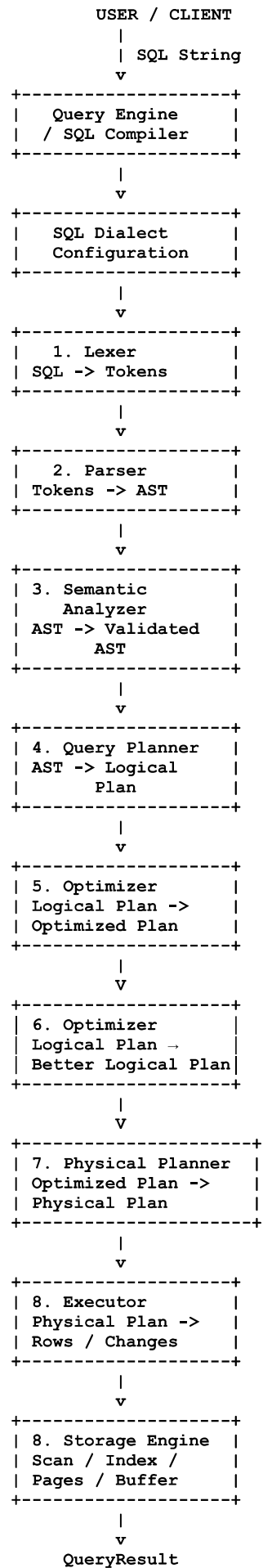
plan/
├── Plan.java, TablePlan.java, SelectPlan.java, ProjectPlan.java
├── Scan.java, TableScan.java, SelectScan.java, ProjectScan.java
└── QueryPlanner, BasicQueryPlanner, UpdatePlanner, BasicUpdatePlanner

```

```

engine/
├── BasicPlanner.java
└── BasicQueryEngine.j

```



|

Phase 1 — Lexical Analysis:

From SQL Text to Tokens

A lexer turns raw text into a flat sequence of tokens — the smallest meaningful pieces of the language (keywords, identifiers, literals, operators, punctuation) — with no notion yet of how they combine into a statement. That structural question is entirely the parser's job (Part 2).

1.1. Source Location

Every token and AST node should carry source information.

```
public record SourceLocation(  
    int offset,  
    int line,  
    int column  
) {}
```

This enables:

query.sql:4:18

```
SELECT *  
FROM users  
WHERE age >=
```

1.3. Token

```
public record Token(  
    TokenType type,  
    String lexeme, ← String text  
    Object literal,  
    SourceLocation location  
) {}
```

literal — What value does the lexeme represent?

Consider:

```
SELECT * FROM users WHERE age >= 18;
```

The lexeme is: **18**

But your query engine eventually wants:
Integer.valueOf(18)

So you could have:

Token(TokenType.NUMBER, "18", 18, ...)	Here: lexeme = "18" literal = 18
---	--

Example2: TRUE Token(TokenType.TRUE, "TRUE", true, ...)	Example3: NULL Token(TokenType.NULL, "NULL", null, ...)
--	--

1.2. TokenType

```
public enum TokenType {  
    // SQL Query Keywords  
    SELECT, FROM, WHERE, AS, DISTINCT  
  
    // Data Manipulation Language (DML)  
    INSERT, INTO, VALUES, UPDATE, SET, DELETE,  
  
    // Data Definition Language (DDL)  
    CREATE, TABLE, DROP, PRIMARY, KEY,  
  
    // Logical Operators  
    AND, OR, NOT,  
  
    // Comparison Operators  
    EQ, NEQ, GT, GTE, LT, LTE,  
  
    // Conditional / Membership Operators  
    IN, IS, NULL, BETWEEN, LIKE, EXISTS,  
  
    // Punctuation / Delimiters  
    STAR, COMMA, LPAREN, RPAREN,  
  
    // Literals  
    IDENTIFIER, NUMBER, STRING,  
  
    // GROUP BY / HAVING  
    GROUP, BY, HAVING,  
  
    // ORDER BY  
    ORDER, ASC, DESC,  
  
    // Pagination  
    LIMIT, OFFSET,  
  
    // JOIN Operations  
    JOIN, INNER, LEFT, RIGHT,  
    FULL, OUTER, CROSS, ON,  
  
    // Set Operations  
    UNION, ALL, INTERSECT, EXCEPT,  
  
    // Common Table Expressions (CTE)  
    WITH, RECURSIVE,  
  
    // CASE Expression  
    CASE, WHEN, THEN, ELSE, END,  
  
    // SQL Data Types  
    INT, BIGINT, DECIMAL,  
    VARCHAR, CHAR, BOOLEAN,  
    DATE, TIMESTAMP,  
  
    // End of input  
    EOF  
}
```

Why not just use lexeme? Because SQL has many values that need interpretation.

Q: Why have offset if we already have line and column?

1.4. Keyword Table

Keep keyword handling outside the lexer implementation.

```
public interface KeywordTable {
    Optional<TokenType> lookup(String text);
}

public final class StandardKeywordTable
    implements KeywordTable {

    private static final Map<String, TokenType> KEYWORDS =
        Map.ofEntries(
            Map.entry("SELECT", TokenType.SELECT),
            Map.entry("FROM", TokenType.FROM),
            Map.entry("WHERE", TokenType.WHERE),
            Map.entry("INSERT", TokenType.INSERT),
            Map.entry("UPDATE", TokenType.UPDATE),
            Map.entry("DELETE", TokenType.DELETE),
            Map.entry("GROUP", TokenType.GROUP),
            Map.entry("BY", TokenType.BY),
            Map.entry("ORDER", TokenType.ORDER),
            Map.entry("LIMIT", TokenType.LIMIT),
            Map.entry("OFFSET", TokenType.OFFSET),
            Map.entry("JOIN", TokenType.JOIN),
            Map.entry("LEFT", TokenType.LEFT),
            Map.entry("RIGHT", TokenType.RIGHT),
            Map.entry("INNER", TokenType.INNER),
            Map.entry("ON", TokenType.ON),
            Map.entry("AS", TokenType.AS),
            Map.entry("AND", TokenType.AND),
            Map.entry("OR", TokenType.OR),
            Map.entry("NOT", TokenType.NOT),
            Map.entry("NULL", TokenType.NULL)
        );

    @Override
    public Optional<TokenType> lookup(String text) {
        return Optional.ofNullable(
            KEYWORDS.get(text.toUpperCase())
        );
    }
}
```

Lexer Input: :

```
SELECT id, name
FROM users
WHERE age >= 18;
```

Output:

```
SELECT
IDENTIFIER(id)
COMMA
IDENTIFIER(name)
FROM
IDENTIFIER(users)
WHERE
IDENTIFIER(age)
GREATER_EQUAL
INTEGER_LITERAL(18)
SEMICOLON
EOF
```

```
public interface SqlLexer {
    List<Token> tokenize();
}
```

1.5. Implement the Lexer

```
public final class SqlLexerImpl {
    private final String source;
    private final KeywordTable keywords;
    final List<Token> tokens = new ArrayList<>();

    private int current;
    private int line = 1;
    private int column = 1;

    /*
     * Location where the current token started.
     *
     * These are important because after advance() has consumed
     * characters, current/line/column point to the NEXT character.
     */
    private int tokenStartOffset;
    private int tokenStartLine;
    private int tokenStartColumn;

    public SqlLexerImpl(String source, KeywordTable keywords) {
        this.source = source;
        this.keywords = keywords;
    }

    public List<Token> tokenize() {
        while (!isAtEnd()) {
            //Remember where this token starts.
            tokenStartOffset = current;
            tokenStartLine = line;
            tokenStartColumn = column;

            scanToken();
        }
        tokens.add(new Token(
            TokenType.EOF,
            "",
            null,
            new SourceLocation(current, line, column)
        ));

        return List.copyOf(tokens);
    }
}
```

```
private void scanToken() {
    char c = advance();

    switch (c) {
        // Punctuation
        case '(' -> add(TokenType.LEFT_PAREN);
        case ')' -> add(TokenType.RIGHT_PAREN);
        case ',' -> add(TokenType.COMMA);
        case '.' -> add(TokenType.DOT);
        case ';' -> add(TokenType.SEMICOLON);

        // Arithmetic operators
        case '+' -> add(TokenType.PLUS);
        case '-' -> scanMinusOrComment();
        case '*' -> add(TokenType.STAR);
        case '/' -> scanSlashOrComment();
        case '%' -> add(TokenType.PERCENT);

        // Comparison operators
        case '=' -> add(TokenType.EQUAL);

        case '<' -> add(
            match('=') ?
            TokenType.LESS_EQUAL :
            TokenType.LESS
        );

        case '>' -> add(
            match('=') ?
            TokenType.GREATER_EQUAL :
            TokenType.GREATER
        );

        // String
        case '"' -> string();

        // Whitespace
        case ' ', '\r', '\t', '\n' -> whitespace();

        // Number / Identifier
        default -> {
            if (isDigit(c)) {
                number();
            } else if (isIdentifierStart(c)) {
                identifier();
            } else {
                throw new IllegalArgumentException(
                    "Unexpected character: " + c
                );
            }
        }
    }
}
```

Implement helper methods:

```
advance()    string()    whitespace()
peek()       number()
match()      identifier()
add()
```

// Helper methods:

```
// Consumes and returns the current character.
// It also updates line and column information.
private char advance() {
    char c = source.charAt(current);
    current++;

    if (c == '\n') { line++; column = 1; }
    else column++;
    return c;
}

// Returns the current character without consuming it.
private char peek() {
    if (isAtEnd()) return '\0';
    return source.charAt(current);
}

// Returns the character after the current
// character without consuming it.
private char peekNext() {
    if (current + 1 >= source.length()) return '\0';
    return source.charAt(current + 1);
}

// Consumes the current character if it matches
// expected.
private boolean match(char expected) {
    if (isAtEnd()) return false;
    if (source.charAt(current) != expected) return false;
    advance();
    return true;
}

// =====
// String
// =====
private void string() {
    StringBuilder value = new StringBuilder();
    while (!isAtEnd()) {
        char c = advance();
        // Closing quote.
        if (c == '"') {
            // SQL represents a single quote inside a
            // string using two consecutive single quotes.
            // Example. 'John"s'
            if (peek() == '"') {
                advance();
                value.append('"');
                continue;
            }

            // Normal closing quote.
            String lexeme = source.substring(
                tokenStartOffset,
                current
            );
            tokens.add(new Token(
                STRING, lexeme, value.toString(),
                new SourceLocation(
                    tokenStartOffset,
                    tokenStartLine,
                    tokenStartColumn
                )
            ));
            return;
        }
        value.append(c);
    }
    throw error("Unterminated string literal");
}

// Adds a token using the current
// token's source range.
private void add(TokenType type) {
    String lexeme = source.substring(
        tokenStartOffset,
        current
    );

    tokens.add( new Token(
        type, lexeme, null,
        new SourceLocation(
            tokenStartOffset,
            tokenStartLine,
            tokenStartColumn
        )
    ));
}

// Adds a token with a parsed literal value.
// token's source range.
// Example: source: 123.45
// lexeme: "123.45"
// literal: Double(123.45)
private void add(TokenType type, Object literal) {
    String lexeme = source.substring(
        tokenStartOffset,
        current
    );

    tokens.add( new Token(
        type,
        lexeme,
        literal,
        new SourceLocation(
            tokenStartOffset,
            tokenStartLine,
            tokenStartColumn
        )
    ));
}

// =====
// Number
// =====
private void number() {
    while (isDigit(peek())) advance();

    // Decimal part.
    if (peek() == '.' && isDigit(peekNext())) {
        advance(); // consume '.'
        while (isDigit(peek())) {
            advance();
        }
    }

    String lexeme = source.substring(
        tokenStartOffset,
        current
    );

    Object literal;
    if (lexeme.indexOf('.') >= 0) {
        literal = Double.parseDouble(lexeme);
    } else {
        literal = Long.parseLong(lexeme);
    }
    add(TokenType.NUMBER, literal);
}
```

```
// =====
// Identifier / Keyword
// =====
private void identifier() {
    while (isIdentifierPart(peek())) {
        advance();
    }

    String text = source.substring(
        tokenStartOffset,
        current
    );

    TokenType keywordType = keywords
        .lookup(text)
        .orElse(TokenType.IDENTIFIER);

    add(keywordType);
}

// =====
// Whitespace
// =====
private void whitespace() {
    while (!isAtEnd()) {
        char c = peek();
        if (!isWhitespace(c)) {
            break;
        }

        advance();
    }
}

// =====
// Character classification
// =====
private boolean isAtEnd() {
    return current >= source.length();
}

private boolean isDigit(char c) {
    return c >= '0' && c <= '9';
}

private boolean isIdentifierStart(char c) {
    return
        (c >= 'a' && c <= 'z')
        || (c >= 'A' && c <= 'Z')
        || c == '_';
}

private boolean isIdentifierPart(char c) {
    return isIdentifierStart(c)
        || isDigit(c)
        || c == '$';
}

private boolean isWhitespace(char c) {
    return
        c == ' '
        || c == '\r'
        || c == '\t'
        || c == '\n';
}
```

```
// =====
// Comments
// =====
private void scanMinusOrComment() {
    if (match("-")) {
        while (!isAtEnd() && peek() != '\n') {
            advance();
        }
        return;
    }
    add(TokenType.MINUS);
}

private void scanSlashOrComment() {
    if (match("**")) {
        scanBlockComment();
        return;
    }
    add(TokenType.SLASH);
}

private void scanBlockComment() {
    while (!isAtEnd()) {
        if (peek() == '*' && peekNext() == '/') {
            advance(); // '*'
            advance(); // '/'
            return;
        }
        advance();
    }
    throw error("Unterminated block comment");
}

// =====
// Error
// =====
private IllegalArgumentException error(String
message) {
    return new IllegalArgumentException(
        message
        + " at line "
        + line
        + ", column "
        + column
        + ", offset "
        + current
    );
}
```

Phase 2 — Defining the SQL Language: Grammar and Syntax

2.1. Initial Grammar

sql
::= statement* EOF

statement
::= selectStatement
| insertStatement
| updateStatement
| deleteStatement
| createTableStatement

2.2. SELECT Grammar

selectStatement
::= SELECT
selectList
fromClause?
whereClause?
groupByClause?
havingClause?
orderByClause?
limitClause?
offsetClause?
", "
,"

selectList
::= "*"
| selectItem ("," selectItem)*

selectItem
::= expression alias?

alias
::= AS IDENTIFIER
| IDENTIFIER

2.3. FROM Grammar

fromClause
::= FROM tableReference

tableReference
::= tableFactor joinClause*

tableFactor
::= tableName alias?
| "(" selectStatement ")" alias?

tableName
::= qualifiedName

qualifiedName
::= IDENTIFIER ("." IDENTIFIER)*

2.4. Expression Grammar

expression
::= logicalOr

logicalOr
::= logicalAnd ("OR" logicalAnd)*

logicalAnd
::= logicalNot ("AND" logicalNot)*

logicalNot
::= NOT logicalNot
| comparison

comparison
::= additive
(
comparisonOperator additive
| IS NOT? NULL
| IN "(" inBody ")"
| BETWEEN additive AND additive
| LIKE additive
)*

comparisonOperator
::= "=" | "<>" | "!="
| "<" | "<=" | ">" | ">="

additive
::= multiplicative
(("+" | "-") multiplicative)*

multiplicative
::= unary
(("*" | "/" | "%") unary)*

unary
::= ("+" | "-") unary
| primary

primary
::= literal
| qualifiedColumn
| functionCall
| "(" expression ")"
| caseExpression
| subqueryExpression
| parameter

2.5. Filtering and Aggregation Grammar : WHERE / GROUP / HAVING

whereClause
::= WHERE expression

groupByClause
::= GROUP BY expressionList

havingClause
::= HAVING expression

2.6. JOIN Grammar

joinClause
::= joinType? JOIN tableFactor
joinCondition?

joinType
::= INNER
| LEFT OUTER?
| RIGHT OUTER?
| FULL OUTER?
| CROSS

joinCondition
::= ON expression

2.7. Operator Precedence

Input: a + b * c
must become:

```
+
/ a *
/ b c
```

Therefore: a + (b * c)
is represented in the AST.

2.8. ORDER / LIMIT / OFFSET

orderByClause
::= ORDER BY orderItem ("," orderItem)*

orderItem
::= expression (ASC | DESC)?

limitClause
::= LIMIT INTEGER_LITERAL

offsetClause
::= OFFSET INTEGER_LITERAL

Phase 3 — The SQL Domain Model: Representing Tables, Rows, and Types

This phase answers: **What does the data look like?**

```
// catalog/DataType.java
public enum DataType {
    INT, BIGINT, BOOLEAN, VARCHAR, DECIMAL, DATE, TIMESTAMP
}
```

```
// catalog/Column.java
public record Column(String name, DataType type, boolean nullable) { }
```

```
// catalog/TableSchema.java
public record TableSchema(String tableName, List<Column> columns) {
    public List<String> fieldNames() {
        return columns.stream().map(Column::name).toList();
    }

    public TableSchema project(List<String> fieldNames) {
        List<Column> projected = columns.stream().filter(c -> fieldNames.contains(c.name())).toList();
        return new TableSchema(tableName, projected);
    }
}
```

```
// A row is deliberately a thin wrapper over a Map for this guide's purposes
// companion Storage Engine guide for the REAL binary,
// on-disk row representation this
// in-memory shape stands in for at execution time.
public final class Row {
    private final Map<String, Object> values;

    public Row(Map<String, Object> values) { this.values = Map.copyOf(values); }

    public Object get(String column) { return values.get(column); }
    public Set<String> columnNames() { return values.keySet(); }
}
```

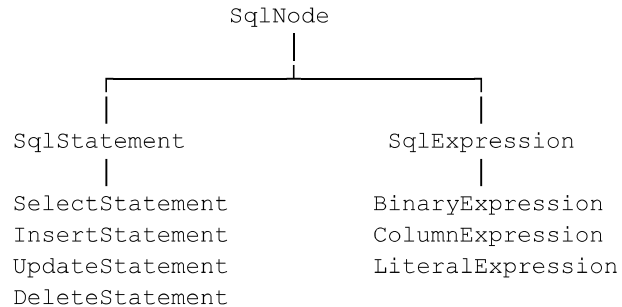
```
// catalog/Catalog.java
public interface Catalog {
    void createTable(TableSchema schema);
    void dropTable(String tableName);
    TableSchema getTable(String tableName);
    boolean tableExists(String tableName);
    List<TableSchema> listTables();
}
```

Phase 4 — The SQL AST: Representing SQL as a Structured Tree

The parser's entire output is one value: a `SqlStatement`. Making it a sealed interface is the single most consequential design decision in this guide — every later dispatch (executors in Part 3, the planner in Part 4) gets compiler-enforced exhaustiveness from this one choice:

4.1. The Root SQL Node

```
// ast/SqlStatement.java
public sealed interface SqlStatement
    extends SqlNode
    permits SelectStatement,
           InsertStatement,
           UpdateStatement,
           DeleteStatement,
           CreateTableStatement,
           DropTableStatement {
}
```



Why have `SqlNode`?

It gives you a single abstraction for "anything that belongs to the SQL AST."

For example:

```
public sealed interface SqlNode
    permits SqlStatement,
           Expression,
           TableReference,
           OrderByItem {
    SourceLocation location();
}
```

Then:

```
public sealed interface SqlExpression
    extends SqlNode
    permits
        LiteralExpression,
        ColumnExpression,
        BinaryExpression {
}
```

And:

```
public record ColumnExpression(
    String name
) implements SqlExpression {
}
```

Now you can write code that works with any AST node:

```
void visit(SqlNode node) {
    // inspect/process any SQL AST node
}
```

But `SqlNode` doesn't necessarily need to contain anything

It can initially be just a marker interface:

```
public interface SqlNode {
}
```

Later, you might give it common functionality, for example:

```
public interface SqlNode {
    SourceLocation location();
}
```

where `SourceLocation` tells you where in the original SQL the node came from.

For example:

```
SELECT name FROM users WHERE age > 18
      ↑
```

The `ColumnExpression("name")` could know that it originated around a particular position in the source SQL. That's useful for parser errors and diagnostics.

4.2. The Implementations of SqlStatement AST:

4.2.1. SELECT AST

Simple Select Statement:

```
record SelectStatement(  
    List<String> columns,  
    String table,  
    Expression where  
) implements SqlStatement { }
```

Or more complex:

```
public record SelectStatement(  
    boolean distinct,  
    List<SelectItem> selectItems,  
    TableReference from,  
    Expression where,  
    List<Expression> groupBy,  
    Expression having,  
    List<OrderByItem> orderBy,  
    Integer limit,  
    Integer offset,  
    SourceLocation location  
) implements SqlStatement {  
}
```

SELECT Item

```
public record SelectItem(  
    Expression expression,  
    String alias,  
    SourceLocation location  
) implements SqlNode {  
}
```

Supports:

```
SELECT id  
SELECT id AS user_id  
SELECT COUNT(*) AS total
```

4.2.3. Column AST

```
public record ColumnReference(  
    String qualifier,  
    String name,  
    SourceLocation location  
) implements Expression {  
}
```

For: "id" use:

- qualifier = null
- name = id

For: u.id use:

- qualifier = u
- name = id

For arbitrary qualification, use **QualifiedName**

4.2.2. Table AST

```
public sealed interface TableReference  
    extends SqlNode  
    permits TableName,  
        Join,  
        SubqueryTable {  
}
```

```
public record TableName(  
    QualifiedName name,  
    String alias,  
    SourceLocation location  
) implements TableReference { }
```

```
public record QualifiedName(  
    List<String> parts  
) { }
```

4.2.4. JOIN AST

```
public record Join(  
    TableReference left,  
    JoinType type,  
    TableReference right,  
    Expression condition,  
    SourceLocation location  
) implements TableReference {  
}
```

```
public enum JoinType {  
    INNER, LEFT, RIGHT, FULL, CROSS  
}
```

4.2.5. Literals

```
public sealed interface LiteralExpression  
    extends Expression  
    permits IntegerLiteral,  
        DecimalLiteral,  
        StringLiteral,  
        BooleanLiteral,  
        NullLiteral {  
}
```

Example:

```
public record IntegerLiteral(  
    long value,  
    SourceLocation location  
) implements LiteralExpression {  
}
```

4.2.6. Binary Expression

```
public record BinaryExpression(
    Expression left,
    BinaryOperator operator,
    Expression right,
    SourceLocation location
) implements Expression {
}
```

```
public enum BinaryOperator {
    ADD, SUBTRACT,
    MULTIPLY, DIVIDE, MODULO,

    EQUAL, NOT_EQUAL,
    LESS, LESS_EQUAL,
    GREATER, GREATER_EQUAL,

    AND, OR,
    LIKE
}
```

4.2.8. Function Calls

```
public record FunctionCallExpression(
    String name,
    List<Expression> arguments,
    boolean distinct,
    SourceLocation location
) implements Expression {}
```

Examples:

```
COUNT(*) | COUNT(DISTINCT user_id)
SUM(amount | AVG(salary)
LOWER(name)
COALESCE(name, 'Unknown')
```

4.2.10. NULL / IN/ BETWEEN

deleted_at IS NULL

```
public record NullCheckExpression(
    Expression expression,
    boolean negated,
    SourceLocation location
) implements Expression {}
```

IN: id IN (1, 2, 3)

```
public record InExpression(
    Expression value,
    List<Expression> values,
    boolean negated,
    SourceLocation location
) implements Expression {}
```

BETWEEN: age BETWEEN 18 AND 60

```
public record BetweenExpression(
    Expression value,
    Expression lower,
    Expression upper,
    boolean negated,
    SourceLocation location
) implements Expression {}
```

4.2.7. Unary Expression

```
public record UnaryExpression(
    UnaryOperator operator,
    Expression expression,
    SourceLocation location
) implements Expression {
}
```

```
public enum UnaryOperator {
    PLUS,
    MINUS,
    NOT
}
```

4.2.9. CASE

```
CASE
    WHEN age >= 18 THEN 'adult'
    ELSE 'minor'
END
```

AST:

```
public record CaseExpression(
    List<WhenClause> clauses,
    Expression elseExpression,
    SourceLocation location
) implements Expression {}
```

4.2.11. CREATE TABLE:

```
public record CreateTableStatement(
    TableName table,
    List<ColumnDefinition> columns,
    SourceLocation location
) implements SqlStatement {}
```

Example:

```
CREATE TABLE users (
    id INT,
    name VARCHAR(100),
    active BOOLEAN
);
```

4.2.12. INSERT:

```
public record InsertStatement(
    TableName table,
    List<String> columns,
    List<List<Expression>> values,
    SourceLocation location
) implements SqlStatement {}
```

Example:

```
INSERT INTO users (id, name)
VALUES
    (1, 'A'),
    (2, 'B');
```

4.2.13. UPDATE:

```
public record UpdateStatement(
    TableName table,
    List<Assignment> assignments,
    Expression where,
    SourceLocation location
) implements SqlStatement {}
```

Example:

```
UPDATE users
SET name = 'John',
    age = age + 1
WHERE id = 10;
```

4.2.15. Subqueries:

Support:

```
SELECT *
FROM users
WHERE id IN (
    SELECT user_id
    FROM orders
);
```

Represent:

```
Select
Filter
InExpression
Subquery
Select
```

4.2.17. Set Operations:

Support:

```
SELECT id FROM users
UNION
SELECT id FROM admins;
```

AST:

```
public record SetOperation(
    SqlStatement left,
    SetOperator operator,
    boolean all,
    SqlStatement right,
    SourceLocation location
) implements SqlStatement {}
```

```
public enum SetOperator {
    UNION,
    INTERSECT,
    EXCEPT
}
```

4.2.14. DELETE:

```
DELETE FROM users WHERE id = 10;
```

```
public record DeleteStatement(
    TableName table,
    Expression where,
    SourceLocation location
) implements SqlStatement {}
```

4.2.16. CTEs:

Support:

```
WITH active_users AS (
    SELECT *
    FROM users
    WHERE active = true
)
SELECT *
FROM active_users;
```

AST:

```
public record CommonTableExpression(
    String name,
    SelectStatement query,
    SourceLocation location
) implements SqlNode {}
```

4.2.18. Window Functions:

Later Support:

```
SELECT
    name,
    department_id,
    ROW_NUMBER() OVER (
        PARTITION BY department_id
        ORDER BY salary DESC
    ) AS rank
FROM employees;
```

AST:

```
public record WindowExpression(
    Expression function,
    List<Expression> partitionBy,
    List<OrderByItem> orderBy,
    WindowFrame frame,
    SourceLocation location
) implements Expression {}
```

Phase 5 — Recursive-Descent Parsing: From Tokens to SQL AST (SqlStatement)

Recursive descent means: one method per grammar rule (§8), each method consuming tokens and calling other rule-methods for sub-rules, mirroring the grammar's own structure almost line for line.

```
public interface SqlParser {  
    List<SqlStatement> parse();  
}
```

TokenStream:

Keep token navigation in one place.

```
public interface TokenStream {  
    Token peek();  
    Token peek(int distance);  
    Token previous();  
    Token advance();  
    boolean check(TokenType type);  
    boolean match(TokenType... types);  
    Token consume(  
        TokenType type,  
        String message  
    );  
    boolean isAtEnd();  
}
```

Statement Dispatcher

As the parser grows, split:

SelectParser, ExpressionParser, DmlParser, DdlParser

```
public final class SqlParser {  
    private SqlStatement parseStatement() {  
        return switch (tokens.peek().type()) {  
            case SELECT -> parseSelectStatement();  
            case INSERT -> parseInsertStatement();  
            case UPDATE -> parseUpdateStatement();  
            case DELETE -> parseDeleteStatement();  
            case CREATE -> parseCreateStatement();  
  
            default -> throw error(  
                tokens.peek(), "Expected SQL statement"  
            );  
        };  
    }  
}
```

TokenStream simply wraps the `List<Token>` produced by the lexer. **TokenStream** focuses on token navigation.

TokenStream is not strictly required. Your parser can work directly with `List<Token>`. Its purpose is to make token navigation a separate responsibility from SQL grammar parsing.

```
public final class TokenStream {  
    private final List<Token> tokens;  
    private int current = 0;  
  
    public TokenStream(List<Token> tokens) {  
        this.tokens = List.copyOf(tokens);  
    }  
  
    public Token peek() {  
        return tokens.get(current);  
    }  
  
    public Token previous() {  
        return tokens.get(current - 1);  
    }  
  
    public Token advance() {  
        if (!isAtEnd()) {  
            current++;  
        }  
        return previous();  
    }  
  
    public boolean check(TokenType type) {  
        if (isAtEnd()) {  
            return type == TokenType.EOF;  
        }  
        return peek().type() == type;  
    }  
}
```

```
public boolean match(TokenType type) {  
    if (!check(type)) {  
        return false;  
    }  
    advance();  
    return true;  
}  
  
public Token consume(TokenType type, String message) {  
    if (check(type)) {  
        return advance();  
    }  
    throw new SqlParseException(message, peek().location());  
}  
  
public boolean isAtEnd() {  
    return peek().type() == TokenType.EOF;  
}
```

The SqlParser

```
// parser/SqlParser.java
public final class SqlParser {
    private final TokenStream tokens;
    private int position = 0;

    public SqlParser(List<Token> tokens) {this.tokens = new TokenStream(tokens);}

    public SqlStatement parseStatement() {
        return switch (peek().type()) {
            case SELECT -> parseSelectStatement();
            case INSERT -> parseInsertStatement();
            case UPDATE -> parseUpdatStatemente();
            case DELETE -> parseDeleteStatement();
            case CREATE -> parseCreateTableStatement();
            case DROP -> parseDropTableStatement();
            default -> throw new IllegalArgumentException("Unexpected token: " + peek());
        };
    }

    // ---- Token stream helpers ----
    private Token peek() { return tokens.get(position); }

    private Token advance() { return tokens.get(position++); }

    private Token expect(TokenType type) {
        Token token = advance();
        if (token.type() != type) throw new IllegalArgumentException("Expected " + type + " but found " + token);
        return token;
    }
    private boolean check(TokenType type) { return peek().type() == type; }
}
```

peek/advance/expect are the entire vocabulary a recursive-descent parser needs — look at the next token without consuming it, consume the next token, or consume it while asserting it's the type you expected. Every grammar rule below is built from exactly these three operations

5.1. SELECT Parser

5.1.1. Parse SelectStatement:

```
// SqlParser.java, continued.....
private SelectStatement parseSelectStatement() {
    Token start = tokens.consume(TokenType.SELECT, "Expected SELECT");

    // SELECT DISTINCT ...
    boolean distinct = tokens.match(TokenType.DISTINCT);

    // SELECT [DISTINCT] <select-list>
    List<SelectItem> items = parseSelectList();

    // FROM is optional in standard SQL parser designs.
    // Examples:
    //   SELECT 1
    //   SELECT 1 + 2
    //   SELECT NOW()
    TableReference from = null;
    if (tokens.match(TokenType.FROM)) from = parseFromClause();

    TableReference from = parseFromClause();

    // where
    Expression where = null;
    if (tokens.match(TokenType.WHERE)) where = parseExpression();

    // GROUP BY
    List<Expression> groupBy = List.of();
    if (tokens.match(TokenType.GROUP)) {
        tokens.consume(TokenType.BY, "Expected BY after GROUP");
        groupBy = expressionList();
    }

    // HAVING
    Expression having = null;
    if (tokens.match(TokenType.HAVING)) having = parseExpression();

    // ORDER BY
    List<OrderByItem> orderBy = List.of();
    if (tokens.match(TokenType.ORDER)) {
        tokens.consume(TokenType.BY, "Expected BY after ORDER");
        orderBy = orderByList();
    }

    // LIMIT
    Integer limit = null;
    if (tokens.match(TokenType.LIMIT)) limit = parseInteger();

    // OFFSET
    Integer offset = null;
    if (tokens.match(TokenType.OFFSET)) offset = parseInteger();

    return new SelectStatement(distinct, items, from, where, groupBy, having, orderBy, limit, offset, start.location());
}
```

```
public record SelectItem(
    Expression expression,
    String alias,
    SourceLocation location
) {}
```

```
private List<SelectItem> parseSelectList() {
    List<SelectItem> items = new ArrayList<>();

    do {
        items.add(parseSelectItem());
    } while (tokens.match(TokenType.COMMA));

    return items;
}
```

// SqlParser.java, continued.....

```
private SelectItem parseSelectItem() {
    Token start = tokens.peek();

    // SELECT *
    if (tokens.match(TokenType.STAR)) {
        return new SelectItem(
            new StarExpression(start.location()),
            null,
            start.location()
        );
    }

    // SELECT expression
    Expression expression = parseExpression();

    // Optional alias:
    //
    // SELECT name AS user_name
    // SELECT name user_name
    String alias = null;

    if (tokens.match(TokenType.AS)) {
        alias = parseIdentifier();
    } else if (tokens.check(IDENTIFIER)) {
        /*
         * Optional implicit alias:
         *
         * SELECT name user_name
         *
         * Be careful here. Only
         * consume an identifier when it
         * cannot be interpreted as
         * part of the expression.
         */
        alias = parseIdentifier();
    }

    return new SelectItem(
        expression,
        alias,
        start.location()
    );
}
```

```
private TableReference parseFromClause() {
    return parseTableReference();
}

private TableReference parseTableReference() {
    Token start = tokens.peek();
    String tableName = parseIdentifier();
    String alias = null;

    if (tokens.match(TokenType.AS)) {
        alias = parseIdentifier();
    } else if (tokens.check(TokenType.IDENTIFIER)) {
        alias = parseIdentifier();
    }

    TableReference table = new TableReference(
        tableName,
        alias,
        start.location()
    );

    return parseJoins(table);
}

private TableReference parseJoins(TableReference left) {
    while (isJoinStart()) {
        JoinType joinType = parseJoinType();

        TableReference right =
            parseTableReferenceWithoutJoins();

        Expression condition = null;

        if (tokens.match(TokenType.ON)) {
            condition = parseExpression();
        }

        left = new JoinReference(
            left,
            joinType,
            right,
            condition,
            left.location()
        );
    }

    return left;
}
```

5.1.2. JOIN support

```
private TableReference parseJoins(
    TableReference left) {
    while (isJoinStart()) {
        JoinType joinType = parseJoinType();

        TableReference right =
            parseTableReferenceWithoutJoins();

        Expression condition = null;

        if (tokens.match(TokenType.ON)) {
            condition = parseExpression();
        }

        left = new JoinReference(
            left, joinType,
            right, condition,
            left.location()
        );
    }
    return left;
}
```

```
private boolean isJoinStart() {
    return tokens.check(TokenType.JOIN)
        || tokens.check(TokenType.INNER)
        || tokens.check(TokenType.LEFT)
        || tokens.check(TokenType.RIGHT)
        || tokens.check(TokenType.FULL)
        || tokens.check(TokenType.CROSS);
}
```

```
private JoinType parseJoinType() {
    if (tokens.match(TokenType.JOIN))
        return JoinType.INNER;

    if (tokens.match(TokenType.INNER)) {
        tokens.consume(TokenType.JOIN,
            "Expected JOIN after INNER");
        return JoinType.INNER;
    }

    if (tokens.match(TokenType.LEFT)) {
        tokens.match(TokenType.OUTER);
        tokens.consume(TokenType.JOIN,
            "Expected JOIN after LEFT");
        return JoinType.LEFT;
    }

    if (tokens.match(TokenType.RIGHT)) {
        tokens.match(TokenType.OUTER);
        tokens.consume(TokenType.JOIN,
            "Expected JOIN after RIGHT");
        return JoinType.RIGHT;
    }

    if (tokens.match(TokenType.FULL)) {
        tokens.match(TokenType.OUTER);
        tokens.consume(TokenType.JOIN,
            "Expected JOIN after FULL");
        return JoinType.FULL;
    }

    if (tokens.match(TokenType.CROSS)) {
        tokens.consume(TokenType.JOIN,
            "Expected JOIN after CROSS");
        return JoinType.CROSS;
    }

    throw error(
        tokens.peek(), "Expected JOIN type"
    );
}
```

```
private TableReference
parseTableReferenceWithoutJoins() {
    Token start = tokens.peek();
    String tableName = parseIdentifier();
    String alias = null;

    if (tokens.match(TokenType.AS)) {
        alias = parseIdentifier();
    } else if (tokens.check(TokenType.IDENTIFIER)) {
        alias = parseIdentifier();
    }

    return new TableReference(
        tableName,
        alias,
        start.location()
    );
}
```

5.1.3. Expression list:

This is used by:

```
GROUP BY department, status, country
```

and can also be reused elsewhere.

```
private List<Expression> expressionList() {
    List<Expression> expressions = new ArrayList<>();

    expressions.add(parseExpression());

    while (tokens.match(TokenType.COMMA)) {
        expressions.add(parseExpression());
    }

    return expressions;
}
```

5.1.4. ORDER BY

ORDER BY name
ORDER BY name ASC
ORDER BY age DESC
ORDER BY department ASC, salary DESC

```
private List<OrderByItem> orderByList() {
    List<OrderByItem> items = new ArrayList<>();

    do {
        items.add(parseOrderByItem());
    } while (tokens.match(TokenType.COMMA));
    return items;
}
```

```
private OrderByItem parseOrderByItem() {
    Token start = tokens.peek();
    Expression expression = parseExpression();
    OrderDirection direction = OrderDirection.ASC;

    if (tokens.match(TokenType.ASC)) {
        direction = OrderDirection.ASC;
    } else if (tokens.match(TokenType.DESC)) {
        direction = OrderDirection.DESC;
    }

    return new OrderByItem(
        expression,
        direction,
        start.location()
    );
}
```

You could define:

```
public enum OrderDirection {
    ASC,
    DESC
}
```

and:

```
public record OrderByItem(
    Expression expression,
    OrderDirection direction,
    SourceLocation location
) {}
```

5.1.5. Integer parsing for LIMIT/OFFSET

```
private Integer parseInteger() {
    Token token = tokens.consume(
        TokenType.NUMBER,
        "Expected integer"
    );

    Object literal = token.literal();

    if (literal instanceof Integer value) {
        return value;
    }

    if (literal instanceof Long value) {
        if (value > Integer.MAX_VALUE ||
            value < Integer.MIN_VALUE) {
            throw error(
                token,
                "Integer value out of range"
            );
        }
        return value.intValue();
    }
}
```

```
if (literal instanceof Number value) {
    double number = value.doubleValue();
    if (number != Math.floor(number)) {
        throw error(token, "Expected integer");
    }

    if (number > Integer.MAX_VALUE ||
        number < Integer.MIN_VALUE) {
        throw error(token, "Integer value out of range");
    }
    return (int) number;
}

// Fallback if lexer stores NUMBER literal as null.
try {
    return Integer.parseInt(token.lexeme());
} catch (NumberFormatException e) {
    throw error(
        token,
        "Expected integer"
    );
}
```

Identifier parsing

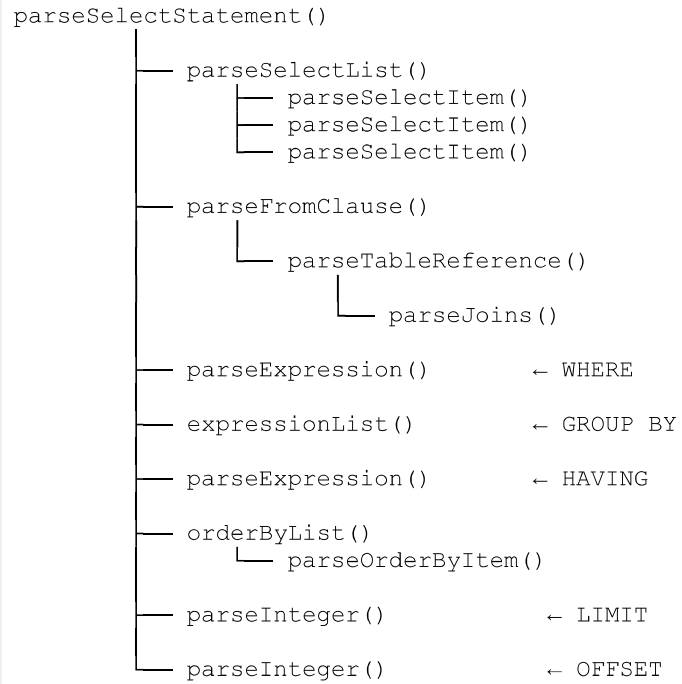
```
private String parseIdentifier() {
    Token token = tokens.consume(
        TokenType.IDENTIFIER,
        "Expected identifier"
    );

    return token.lexeme();
}
```

So a query such as:

```
SELECT
  u.id,
  u.name AS user_name,
  COUNT(*) AS order_count
FROM users u
LEFT JOIN orders o
  ON u.id = o.user_id
WHERE u.active = TRUE
GROUP BY u.id, u.name
HAVING COUNT(*) > 5
ORDER BY order_count
DESC
LIMIT 20
OFFSET 10
```

will flow through:



5.2. Expression Parser

The precedence in your code is: **OR** → **AND** → **NOT** → **comparison**

Start with recursive descent:

```
private Expression expression() {  
    return logicalOr();  
}
```

Whenever the parser needs to parse an expression, it starts at the lowest-precedence level: OR.

```
private Expression logicalOr() {  
    Expression left = logicalAnd();  
  
    while (tokens.match(TokenType.OR)) {  
        Expression right = logicalAnd();  
  
        left = new BinaryExpression(  
            left,  
            BinaryOperator.OR,  
            right,  
            left.location()  
        );  
    }  
    return left;  
}
```

Why does it call logicalAnd()?

Because AND has higher precedence than OR.

Consider: **a OR b AND c**

We want: **a OR (b AND c)**

not: **(a OR b) AND c**

So logicalOr() first asks logicalAnd() to parse its operands.

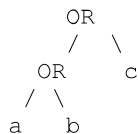
Why use a while loop?

allows multiple ORs. **a OR b OR c**

the parser does:

```
left = a  
  
OR found  
right = b  
left = (a OR b)  
  
OR found  
right = c  
left = ((a OR b) OR c)
```

So the resulting AST is roughly:



This is called left associativity.

logicalAnd()

```
private Expression logicalAnd() {  
    Expression left = logicalNot();  
  
    while (tokens.match(TokenType.AND)) {  
        Expression right = logicalNot();  
  
        left = new BinaryExpression(  
            left,  
            BinaryOperator.AND,  
            right,  
            left.location()  
        );  
    }  
  
    return left;  
}
```

The important part is: Expression left = logicalNot();
That means AND sits above NOT in precedence.

For: **a AND NOT b**
the parser sees:

```
logicalAnd()  
|  
+-- logicalNot() → a  
|  
+-- AND  
|  
+-- logicalNot() → NOT b
```

Result: **a AND (NOT b)**

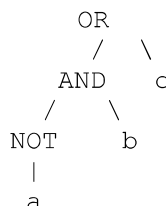
logicalNot()

```
private Expression logicalNot() {  
    if (tokens.match(TokenType.NOT)) {  
        Expression operand = logicalNot();  
  
        return new UnaryExpression(  
            UnaryOperator.NOT,  
            operand,  
            operand.location()  
        );  
    }  
  
    return comparison();  
}
```

This one is slightly different. **NOT** is a unary operator, meaning it operates on one expression.

Putting everything together:

NOT a AND b OR c



5.3. Comparison Parser

```
private Expression comparison() {
    Expression left = additive();

    if (tokens.match(
        TokenType.EQUAL,
        TokenType.NOT_EQUAL,
        TokenType.LESS,
        TokenType.LESS_EQUAL,
        TokenType.GREATER,
        TokenType.GREATER_EQUAL
    )) {

        Token operator = tokens.previous();
        Expression right = additive();

        return new BinaryExpression(
            left,
            mapOperator(operator),
            right,
            operator.location()
        );
    }

    if (tokens.match(TokenType.IS)) {
        boolean negated =
            tokens.match(TokenType.NOT);
        tokens.consume(TokenType.NULL,
            "Expected NULL");

        return new NullCheckExpression(
            left,
            negated,
            left.location()
        );
    }

    // IN / BETWEEN / LIKE...

    return left;
}
```

What is additive()?

additive() is the parser level responsible for + and - arithmetic.

For example:

```
a + b
a + b - c
```

Implementation:

```
private Expression additive() {
    Expression left = multiplicative();

    while(tokens.match(PLUS, MINUS)) {
        Token operator = tokens.previous();
        Expression right = multiplicative();

        left = new BinaryExpression(
            left, mapOperator(operator),
            right, operator.location()
        );
    }
    return left;
}
```

Handle:

=	IS NULL
<>	IS NOT NULL
!=	IN
<	NOT IN
<=	BETWEEN
>	NOT
>=	LIKE
	NOT LIKE

5.4. Pratt Parser Option

As expression syntax becomes large, a Pratt parser can replace the precedence methods.

Example binding powers:

OR	10
AND	20
=	30
<	30
>	30
+	40
-	40
*	50
/	50
%	50

Recommended evolution:

Initial implementation: **Recursive descent**

Advanced implementation: **Pratt parser**

Why does comparison() call additive()?

Consider: **a + 5 > b * 2**

You want the AST to mean:

```
(a + 5) > (b * 2)
```

not:

```
a + (5 > b) * 2
```

So comparison() delegates the operands to additive():

```
comparison
|
+----- additive → a + 5
|
>
|
+----- additive → b * 2
```

Result:

```
      >
     / \
    (+)  (*)
   / \  / \
  a  5 b  2
```

The WHERE Clause: A Small Expression AST

```
// ast/Expression.java
public sealed interface Expression permits Comparison, And, Or { }

public record Comparison(String column, String operator, Object value) implements Expression { }
public record And(Expression left, Expression right) implements Expression { }
public record Or(Expression left, Expression right) implements Expression { }
```

```
// SqlParser.java, continued - implements
expression := comparison | expression AND
expression | expression OR expression
private Expression parseWhereClause() {
    expect(TokenType.WHERE);
    return parseExpression();
}

private Expression parseExpression() {
    Expression left = parseComparison();
    while (check(TokenType.AND) || check(TokenType.OR))
    {
        boolean isAnd = check(TokenType.AND);
        advance();
        Expression right = parseComparison();
        left = isAnd ? new And(left, right) : new Or(left, right);
    }
    return left;
}

private Comparison parseComparison() {
    String column = expect(TokenType.IDENTIFIER).text();
    String operator = parseOperator();
    Object value = parseLiteral();
    return new Comparison(column, operator, value);
}

private String parseOperator() {
    Token token = advance();
    return switch (token.type()) {
        case EQ -> "="; case NEQ -> "!="; case GT -> ">";
        case GTE -> ">="; case LT -> "<"; case LTE -> "<=";
        default -> throw new
        IllegalArgumentException("Expected a comparison
        operator but found " + token);
    };
}
```

salary > 50000 AND department = 'Engineering' parses into **And(Comparison("salary", ">", 50000), Comparison("department", "=", "Engineering"))** — a tree shape §13's evaluator walks recursively, matching the sealed interface's own shape exactly.

Implementing the Expression Evaluator

```
// ast/ExpressionEvaluator.java
public final class ExpressionEvaluator {
    public static boolean matches(Row row, Expression expression) {
        return switch (expression) {
            case Comparison c -> evaluateComparison(row.get(c.column()), c);
            case And a -> matches(row, a.left()) && matches(row, a.right());
            case Or o -> matches(row, o.left()) || matches(row, o.right());
        };
    }

    static boolean evaluateComparison(Object actual, Comparison c) {
        int cmp = compare(actual, c.value());
        return switch (c.operator()) {
            case "=" -> Objects.equals(actual, c.value());
            case "!=" -> !Objects.equals(actual, c.value());
            case ">" -> cmp > 0;
            case ">=" -> cmp >= 0;
            case "<" -> cmp < 0;
            case "<=" -> cmp <= 0;
            default -> throw new IllegalStateException("Unsupported operator: " + c.operator());
        };
    }

    @SuppressWarnings("unchecked")
    private static int compare(Object actual, Object expected) {
        if (actual instanceof Comparable<?> && expected != null) {
            return ((Comparable<Object>) actual).compareTo(expected);
        }
        return 0;
    }
}
```

5.5. Other Parsing Statements...

```
private InsertStatement parseInsertStatement() {
    expect(TokenType.INSERT);
    expect(TokenType.INTO);
    String table = expect(IDENTIFIER).text();

    expect(TokenType.LPAREN);
    List<String> columns = parseIdentifierList();
    expect(TokenType.RPAREN);

    expect(TokenType.VALUES);
    expect(TokenType.LPAREN);
    List<Object> values = parseValueList();
    expect(TokenType.RPAREN);

    return new InsertStatement(table, columns, values);
}

private List<Object> parseValueList() {
    List<Object> values = new ArrayList<>();
    values.add(parseLiteral());
    while (check(TokenType.COMMA)) {
        advance();
        values.add(parseLiteral());
    }
    return values;
}

private DeleteStatement parseDelete() {
    expect(TokenType.DELETE);
    expect(TokenType.FROM);
    String table = expect(IDENTIFIER).text();
    Expression where = check(WHERE) ?
        parseWhereClause() : null;
    return new DeleteStatement(table, where);
}

private CreateTableStatement parseCreateTable() {
    expect(TokenType.CREATE);
    expect(TokenType.TABLE);
    String table = expect(IDENTIFIER).text();

    expect(TokenType.LPAREN);
    List<Column> columns = new ArrayList<>();
    columns.add(parseColumnDefinition());
    while (check(TokenType.COMMA)) {
        advance();
        columns.add(parseColumnDefinition());
    }
    expect(TokenType.RPAREN);

    return new CreateTableStatement(
        new TableSchema(table, columns));
}

private DropTableStatement parseDropTable() {
    expect(TokenType.DROP);
    expect(TokenType.TABLE);
    String table = expect(IDENTIFIER).text();
    return new DropTableStatement(table);
}
```

```
Map<String, Object> parseAssignment() {
    String column = expect(IDENTIFIER).text();
    expect(TokenType.EQ);
    Object value = parseLiteral();
    return Map.of(column, value);
}

private UpdateStatement parseUpdate() {
    expect(TokenType.UPDATE);
    String table = expect(IDENTIFIER).text();
    expect(TokenType.SET);

    Map<String, Object> assignments = new
        LinkedHashMap<>();
    assignments.putAll(parseAssignment());
    while (check(TokenType.COMMA)) {
        advance();
        assignments.putAll(parseAssignment());
    }

    Expression where = check(WHERE) ?
parseWhereClause() : null;
    return new UpdateStatement(table, assignments,
        where);
}

private Column parseColumnDefinition() {
    String name = expect(IDENTIFIER).text();
    DataType type =
        DataType.valueOf(expect(IDENTIFIER).text().toUpperCase());
    boolean isPrimaryKey = false;
    if (check(TokenType.PRIMARY)) {
        advance();
        expect(TokenType.KEY);
        isPrimaryKey = true;
    }
    return new Column(name, type, !isPrimaryKey);
    // a primary key column is implicitly NOT NULL
}
```

Phase 6 — Semantic Analysis: Validating the SQL

Parsing answers:

Is this syntactically valid?

Semantic analysis answers:

Does this SQL make sense against a schema?

Example:

```
SELECT unknown_column  
FROM users;
```

can parse correctly but fail semantic validation.

6.1. Example: syntax vs semantics

Consider:

```
SELECT name, age  
FROM users  
WHERE age > 18;
```

The parser checks things such as:

SELECT → expression list → FROM →
table → WHERE → expression

So this is syntactically valid.

But consider:

```
SELECT name, salary  
FROM users  
WHERE age > 'hello';
```

The syntax may be perfectly valid.

Semantic analysis asks:

Does users.salary exist?
Does users.name exist?
Does users.age exist?
Is age comparable with 'hello'?

If age is **INTEGER** and 'hello' is **VARCHAR**, that's a semantic/type **error**.

6.2. What should your Semantic Analyzer do?

For your TinyDB, I would initially give it these responsibilities:

A. Table resolution

```
SELECT name  
FROM users;
```

Check:

Does table "users" exist?

If not:

Semantic error:
Table 'users' does not exist

B. Column resolution

```
SELECT name, age  
FROM users;
```

Check:

users.name exists?
users.age exists?

C. Column ambiguity

This becomes important with JOINS.

```
SELECT id  
FROM users  
JOIN orders ON users.id = orders.user_id;
```

If both tables contain id → ambiguous.:
The semantic analyzer should report:

Ambiguous column 'id'

6.3. Type checking

Your schema might contain:

```
users
-----
id      INTEGER
name    VARCHAR
age     INTEGER
active  BOOLEAN
salary  DECIMAL
```

Now:

```
SELECT *
FROM users
WHERE age > 18;
```

is valid.

But:

```
SELECT *
FROM users
WHERE age > 'hello';
```

should fail semantic analysis.

You could define a type system such as:

```
public enum DataType {
    INTEGER,
    BIGINT,
    DECIMAL,
    VARCHAR,
    BOOLEAN,
    DATE,
    TIMESTAMP,
    NULL
}
```

6.4. DELETE validation

For:

```
DELETE FROM users
WHERE age > 18;
```

check:

```
users exists
age exists
age is numeric
age > 18 produces BOOLEAN
```

6.5. Operator validation

Semantic analysis also validates operators.

For example:

```
age + 10
```

is reasonable if age is numeric.

Similarly:

```
age LIKE '%20%'
```

would normally be invalid if LIKE requires a string expression and age is numeric.

You could have rules such as:

```
+ → numeric + numeric
= → compatible types
> → comparable types
AND → BOOLEAN AND BOOLEAN
OR → BOOLEAN OR BOOLEAN
NOT → BOOLEAN
LIKE → VARCHAR LIKE VARCHAR
```

6.6. INSERT validation

Consider

```
INSERT INTO users (name, age)
VALUES ('Arpan', 32);
```

Semantic analysis checks:

```
users exists?
name exists?
age exists?
name accepts VARCHAR?
age accepts INTEGER?
```

It could detect:

```
INSERT INTO
users (name, age)
VALUES ('Arpan', 'hello');
```

6.7. UPDATE validation

For:

```
UPDATE users
SET age = 32
WHERE id = 10;
```

check:

```
users exists
age exists
age accepts INTEGER
id exists
id accepts INTEGER
WHERE expression is BOOLEAN
```

6.8. GROUP BY / aggregation

This is where semantic analysis becomes much more valuable.

Suppose:

```
SELECT
    department, name, COUNT(*)
FROM employees
GROUP BY department;
```

Depending on your SQL rules, name is invalid because it isn't:

- in **GROUP BY**, or
- an aggregate expression.

The semantic analyzer can detect this.

6.9. Function validation

Later, if you support:

```
SELECT COUNT(*), AVG(salary)
FROM employees;
```

the analyzer can check:

```
COUNT(*) → valid
AVG(salary) → salary must be numeric
```

Do not hard-code every function into the parser.

```
public interface FunctionRegistry {
    Optional<FunctionDefinition> find(
        String name
    );
}
```

```
public record FunctionDefinition(
    String name,
    List<SqlType> parameterTypes,
    SqlType returnType,
    FunctionKind kind
) {}
```

```
public enum FunctionKind(
    SCALAR,
    AGGREGATE,
    WINDOW
) {}
```

Example:

```
COUNT
SUM
AVG
MIN
MAX
LOWER
UPPER
COALESCE
SUBSTRING
```

6.10. Semantic analyzer needs database metadata

```
SELECT abc FROM xyz;
```

The parser can successfully create an AST even if xyz doesn't exist.

The semantic analyzer needs a Catalog or Schema Registry.

```
public interface Catalog {  
    TableSchema getTable(String tableName);  
    boolean tableExists(String tableName);  
}
```

```
public record TableSchema(  
    String name,  
    List<ColumnSchema> columns  
) {}
```

And:

```
public record ColumnSchema(  
    String name,  
    DataType type,  
    boolean nullable  
) {}
```

Possible implementations:

- InMemoryCatalog
- JdbcCatalog
- PostgresCatalog
- MySqlCatalog
- MockCatalog

6.11 SemanticAnalyzer interface

I'd keep the parser and semantic analyzer separate:

```
public interface SemanticAnalyzer {  
    AnalyzedStatement analyze(SqlStatement statement);  
}
```

For example:

```
public final class SemanticAnalyzerImpl implements SemanticAnalyzer {  
    private final Catalog catalog;  
  
    public SemanticAnalyzerImpl(Catalog catalog) {  
        this.catalog = catalog;  
    }  
  
    @Override  
    public AnalyzedStatement analyze(SqlStatement statement) {  
        // resolve tables  
        // resolve columns  
        // validate types  
        // validate expressions  
        // validate SQL rules  
  
        return ...;  
    }  
}
```

The AnalyzedStatement could contain additional information:

```
public record ResolvedColumn(  
    String tableName,  
    String columnName,  
    DataType type  
) {}
```

Phase 7 — The Six Statement Executors

Before we start building the Planner, we need two small supporting types that are shared across the execution pipeline: `QueryResult` and `TransactionContext`.

These types provide the runtime context needed by the executor and dispatcher while keeping transaction management separate from parsing and query planning.

7.1. Transaction and Query Execution Context

QueryResult:

`QueryResult` represents the outcome of executing a statement.

```
public record QueryResult(List<Row> rows, int affectedRows) {
    public static QueryResult ofRows(List<Row> rows) {
        return new QueryResult(rows, rows.size());
    }

    public static QueryResult ofAffected(int count) {
        return new QueryResult(List.of(), count);
    }
}
```

This gives the execution layer a single, consistent result type regardless of whether a statement produces rows or only modifies data.

TransactionContext :

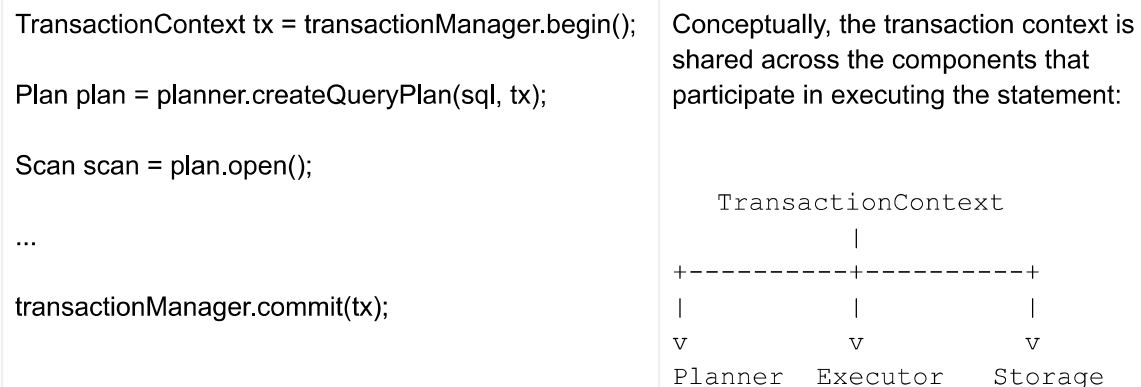
`TransactionContext` represents the transaction under which a statement is executed

```
public final class TransactionContext {
    // Deliberately empty for now. This guide focuses on parsing/planning/execution;
    // real isolation, locking, and rollback belong to a transaction manager, covered
    // in depth by the companion Storage Engine guide's crash-recovery and WAL sections.
    // Until a real TransactionContext is wired in, every statement is its own
    // implicit, auto-committed transaction.

    public static final TransactionContext AUTOCOMMIT = new TransactionContext();
}
```

The important point is that `TransactionContext` is **not a sequential compiler phase** like parsing, analysis, or planning. Instead, it is a **runtime context** that flows through multiple layers of the database engine.

A typical execution flow looks like this:



7.2. A Minimal StorageEngine to Execute Against

The executors need something real underneath them. This in-memory version is the Level 1 stand-in — the companion Storage Engine guide builds the real, page-based, durable implementation of this exact same interface.

```
public interface StorageEngine {
    void createTable(TableSchema schema);
    void dropTable(String tableName);
    void insert(String tableName, Row row);
    List<Row> scan(String tableName);
    void replaceRows(String tableName, List<Row> rows);
    Scan openScan(String tableName); // streaming access — used from Part 5 onward
}
```

```
public final class MemoryStorageEngine implements StorageEngine {
    private final Map<String, List<Row>> tables = new ConcurrentHashMap<>();
    @Override
    public void createTable(TableSchema schema) {
        tables.putIfAbsent(schema.tableName(), new CopyOnWriteArrayList<>());
    }

    @Override
    public void dropTable(String tableName) { tables.remove(tableName); }

    @Override
    public void insert(String tableName, Row row) { tables.get(tableName).add(row); }

    @Override
    public List<Row> scan(String tableName) { return List.copyOf(tables.get(tableName)); }

    @Override
    public void replaceRows(String tableName, List<Row> rows) {
        tables.put(tableName, new CopyOnWriteArrayList<>(rows));
    }

    @Override
    public Scan openScan(String tableName) {
        throw new UnsupportedOperationException("see Part 5");
    }
}
```

replaceRows is a deliberately blunt instrument: **UpdateExecutor/DeleteExecutor** (§17) compute a new row list and swap the whole thing in, rather than mutating individual rows — correct and simple here, and honestly wasteful once a table is large, which is exactly the problem the real page-based storage engine (the companion guide) solves properly.

7.3. The Six Statement Executors

The executors need something real underneath them. This in-memory version is the Level 1 stand-in — the companion Storage Engine guide builds the real, page-based, durable implementation of this exact same interface.

```
public interface StatementExecutor<S extends SqlStatement> {  
    QueryResult execute(S statement, TransactionContext transaction);  
}
```

7.3.1. SelectExecutor

```
public final class SelectExecutor implements StatementExecutor<SelectStatement> {  
    private final StorageEngine storageEngine;  
  
    @Override public QueryResult execute(SelectStatement statement, TransactionContext txn) {  
        List<Row> matching = storageEngine.scan(statement.table()).stream()  
            .filter(row -> statement.where() == null  
                || ExpressionEvaluator.matches(row, statement.where()))  
            .toList();  
  
        if (statement.columns().size() == 1 && statement.columns().get(0).equals("*")) {  
            return QueryResult.ofRows(matching);  
        }  
  
        List<Row> projected = matching.stream()  
            .map(row -> projectColumns(row, statement.columns()))  
            .toList();  
        return QueryResult.ofRows(projected);  
    }  
  
    private Row projectColumns(Row row, List<String> columns) {  
        Map<String, Object> projected = new LinkedHashMap<>();  
        for (String column : columns) {  
            projected.put(column, row.get(column));  
        }  
        return new Row(projected);  
    }  
}
```

7.3.2. InsertExecutor:

```
public final class InsertExecutor implements StatementExecutor<InsertStatement> {  
    private final StorageEngine storageEngine;  
    private final Catalog catalog;  
  
    @Override public QueryResult execute(InsertStatement statement, TransactionContext txn) {  
        TableSchema schema = catalog.getTable(statement.table());  
        if (schema == null) throw new IllegalStateException("No such table: " + statement.table());  
        if (statement.columns().size() != statement.values().size())  
            throw new IllegalArgumentException("Column count does not match value count");  
  
        Map<String, Object> values = new LinkedHashMap<>();  
        for (int i = 0; i < statement.columns().size(); i++) {  
            values.put(statement.columns().get(i), statement.values().get(i));  
        }  
        validateAgainstSchema(schema, values);  
        storageEngine.insert(statement.table(), new Row(values));  
        return QueryResult.ofAffected(1);  
    }  
  
    private void validateAgainstSchema(TableSchema schema, Map<String, Object> values) {  
        for (Column column : schema.columns()) {  
            if (!column.nullable() && values.get(column.name()) == null) {  
                throw new IllegalStateException(column.name() + " cannot be null");  
            }  
        }  
    }  
}
```

7.3.3. UpdateExecutor:

```
public final class UpdateExecutor implements StatementExecutor<UpdateStatement> {
    private final StorageEngine storageEngine;

    @Override
    public QueryResult execute(UpdateStatement statement, TransactionContext txn) {
        List<Row> current = storageEngine.scan(statement.table());
        List<Row> updated = new ArrayList<>();
        int affected = 0;

        for (Row row : current) {
            if (statement.where() == null || ExpressionEvaluator.matches(row, statement.where())) {
                Map<String, Object> newValues = new LinkedHashMap<>();
                for (String column : row.columnNames()) newValues.put(column, row.get(column));
                newValues.putAll(statement.assignments());
                updated.add(new Row(newValues));
                affected++;
            } else {
                updated.add(row);
            }
        }
        storageEngine.replaceRows(statement.table(), updated);
        return QueryResult.ofAffected(affected);
    }
}
```

7.3.4. DeleteExecutor:

```
public final class DeleteExecutor implements StatementExecutor<DeleteStatement> {
    private final StorageEngine storageEngine;

    @Override
    public QueryResult execute(DeleteStatement statement, TransactionContext transaction) {
        List<Row> current = storageEngine.scan(statement.table());
        List<Row> kept = new ArrayList<>();
        int affected = 0;

        for (Row row : current) {
            boolean shouldDelete = statement.where() == null
                || ExpressionEvaluator.matches(row, statement.where());
            if (shouldDelete) {
                affected++;
            } else {
                kept.add(row);
            }
        }
        storageEngine.replaceRows(statement.table(), kept);
        return QueryResult.ofAffected(affected);
    }
}
```

7.3.5. CreateTableExecutor:

```
public final class CreateTableExecutor implements StatementExecutor<CreateTableStatement> {
    private final StorageEngine storageEngine;
    private final Catalog catalog;

    @Override
    public QueryResult execute(CreateTableStatement statement, TransactionContext transaction) {
        if (catalog.tableExists(statement.schema().tableName())) {
            throw new IllegalStateException("Table already exists: " + statement.schema().tableName());
        }
        catalog.createTable(statement.schema());
        storageEngine.createTable(statement.schema());
        return QueryResult.ofAffected(0);
    }
}
```

7.3.6. DropTableExecutor:

```
public final class DropTableExecutor implements StatementExecutor<DropTableStatement> {
    private final StorageEngine storageEngine;
    private final Catalog catalog;

    @Override
    public QueryResult execute(DropTableStatement statement, TransactionContext transaction) {
        if (!catalog.tableExists(statement.table())) {
            throw new IllegalStateException("No such table: " + statement.table());
        }
        catalog.dropTable(statement.table());
        storageEngine.dropTable(statement.table());
        return QueryResult.ofAffected(0);
    }
}
```

7.4. The QueryEngine Dispatcher: Wiring Lexer, Parser, and Executors Together

The executors need something real underneath them. This in-memory version is the Level 1 stand-in — the companion Storage Engine guide builds the real, page-based, durable implementation of this exact same interface.

This is the answer to "how do lexer, parser, and executors actually connect":

```
public final class QueryEngine {
    private final SelectExecutor selectExecutor;
    private final InsertExecutor insertExecutor;
    private final UpdateExecutor updateExecutor;
    private final DeleteExecutor deleteExecutor;
    private final CreateTableExecutor createTableExecutor;
    private final DropTableExecutor dropTableExecutor;

    public QueryEngine(Catalog catalog, StorageEngine storageEngine) {
        this.selectExecutor = new SelectExecutor(storageEngine);
        this.insertExecutor = new InsertExecutor(storageEngine, catalog);
        this.updateExecutor = new UpdateExecutor(storageEngine);
        this.deleteExecutor = new DeleteExecutor(storageEngine);
        this.createTableExecutor = new CreateTableExecutor(catalog, storageEngine);
        this.dropTableExecutor = new DropTableExecutor(catalog, storageEngine);
    }

    public QueryResult run(String sql) {
        List<Token> tokens = new SqlLexer(sql).tokenize();
        SqlStatement statement = new SqlParser(tokens).parseStatement();
        return execute(statement, TransactionContext.AUTOCOMMIT);
    }

    public QueryResult execute(SqlStatement statement, TransactionContext txn) {
        return switch (statement) {
            case SelectStatement s -> selectExecutor.execute(s, txn);
            case InsertStatement s -> insertExecutor.execute(s, txn);
            case UpdateStatement s -> updateExecutor.execute(s, txn);
            case DeleteStatement s -> deleteExecutor.execute(s, txn);
            case CreateTableStatement s -> createTableExecutor.execute(s, txn);
            case DropTableStatement s -> dropTableExecutor.execute(s, txn);
        };
    }
}
```

7.5. Alternative: ANTLR-Generated Parser + the Visitor Pattern

```
// Uses an ANTLR-generated MySQLLexer/MySQLStatementParser instead of a
// hand-written one. ANTLR turns a .g4 grammar file into the lexer,
public class MySqlParser implements IParser {
    MySQLStatementVisitor sqlStatementVisitor;

    public MySqlParser(String sql) {
        MySQLLexer lexer = new MySQLLexer(CharStreams.fromString(sql));
        var parser = new MySQLStatementParser(new CommonTokenStream(lexer));

        sqlStatementVisitor = new MySQLStatementVisitor(parser);
        sqlStatementVisitor.visit(parser.execute());
    }

    @Override
    public QueryData queryCmd() {return (QueryData) sqlStatementVisitor.getValue();}

    @Override
    public Object updateCmd() {return sqlStatementVisitor.getValue(); }
}

// The Visitor walks ANTLR's generated parse tree and extracts exactly
// the fields needed into a domain object -- the ANTLR equivalent of
// this guide's SqlStatement records.
public class MySQLStatementVisitor extends MySQLStatementBaseVisitor {
    private final MySQLStatementParser parser;
    private COMMAND_TYPE commandType;
    private String tableName;

    @Override
    public Object visitCreateTable(MySQLStatementParser.CreateTableContext ctx) {
        commandType = COMMAND_TYPE.CREATE_TABLE;
        return super.visitCreateTable(ctx);
    }

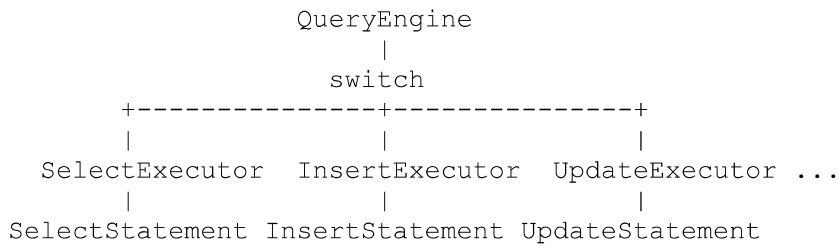
    @Override
    public Object visitTableName(MySQLStatementParser.TableNameContext ctx) {
        this.tableName = ctx.name().getText();
        return super.visitTableName(ctx);
    }
}
```

The pipeline is structurally identical to section 10.1's, just built from generated pieces instead of hand-written ones:

This guide (hand-written):	ANTLR-based (e.g. ShardingSphere):
SQL text	SQL text
SqlLexer (hand-written)	MySQLLexer (generated from a grammar)
SqlParser (hand-written)	MySQLStatementParser (generated) + a
	Visitor subclass that walks its parse
tree	
SqlStatement (a sealed	QueryData / InsertData / ModifyData
interface of records)	(plain domain objects the Visitor builds)
QueryEngine.execute(...)	dispatch by commandType / object type
(an exhaustive switch)	(structurally the same idea, without
	a sealed-type-checked switch

7.5. Understand the Visitor Pattern

1. The current design:



The dispatcher knows about every statement type:

```
return switch (statement) {
  case SelectStatement s -> selectExecutor.execute(s, txn);
  case InsertStatement s -> insertExecutor.execute(s, txn);
  ...
};
```

The nice thing is that your executor classes are strongly separated.

2. With Visitor, move dispatch into the AST:

You could introduce:

```
public interface SqlStatement {
  <R> R accept(SqlStatementVisitor<R> visitor);
}
```

Then:

```
public interface SqlStatementVisitor<R> {
  R visit(SelectStatement statement);
  R visit(InsertStatement statement);
  R visit(UpdateStatement statement);
  R visit>DeleteStatement statement);
  R visit(CreateTableStatement statement);
  R visit(DropTableStatement statement);
}
```

Each statement implements accept():

```
public final class SelectStatement implements SqlStatement {
  // existing fields...

  @Override
  public <R> R accept(SqlStatementVisitor<R> visitor) {
    return visitor.visit(this);
  }
}
```

Likewise:

```
public final class InsertStatement implements SqlStatement {

  @Override
  public <R> R accept(SqlStatementVisitor<R> visitor) {
    return visitor.visit(this);
  }
}
```

And so on.

Now the AST itself knows how to dispatch to the correct visitor method.

3. Make execution a Visitor:

Instead of six separate executor classes, you could make one:

```
public final class ExecutionVisitor implements SqlStatementVisitor<QueryResult> {
    private final StorageEngine storageEngine;
    private final Catalog catalog;
    private final TransactionContext txn;

    @Override
    public QueryResult visit(SelectStatement statement) {
        List<Row> matching = storageEngine.scan(statement.table()).stream()
            .filter(row -> statement.where() == null || ExpressionEvaluator.matches(row, statement.where()))
            .toList();

        if (statement.columns().size() == 1 && statement.columns().get(0).equals("")) {
            return QueryResult.ofRows(matching);
        }

        List<Row> projected = matching.stream()
            .map(row -> projectColumns(row, statement.columns()))
            .toList();

        return QueryResult.ofRows(projected);
    }

    @Override
    public QueryResult visit(InsertStatement statement) {
        TableSchema schema = catalog.getTable(statement.table());
        if (schema == null) throw new IllegalStateException("No such table: " + statement.table());

        if (statement.columns().size() != statement.values().size()) {
            throw new IllegalArgumentException("Column count does not match value count");
        }

        Map<String, Object> values = new LinkedHashMap<>();
        for (int i = 0; i < statement.columns().size(); i++) {
            values.put(statement.columns().get(i), statement.values().get(i));
        }
        validateAgainstSchema(schema, values);

        storageEngine.insert(statement.table(), new Row(values));
        return QueryResult.ofAffected(1);
    }

    @Override
    public QueryResult visit(UpdateStatement statement) { // existing UpdateExecutor logic }

    @Override
    public QueryResult visit(DeleteStatement statement) { // existing DeleteExecutor logic }

    @Override
    public QueryResult visit(CreateTableStatement statement) { // existing CreateTableExecutor logic }

    @Override
    public QueryResult visit(DropTableStatement statement) { // existing DropTableExecutor logic }
}
```

The important part is that double dispatch is happening:

```
statement.accept(visitor);
```

calls:

```
visitor.visit(this);
```

where this is statically the concrete statement type.

So:

```
SelectStatement
-> accept(visitor)
-> visitor.visit(SelectStatement)
```

while:

```
InsertStatement
-> accept(visitor)
-> visitor.visit(InsertStatement)
```

4. QueryEngine becomes much simpler:

```
public final class QueryEngine {
    private final Catalog catalog;
    private final StorageEngine storageEngine;

    public QueryResult run(String sql) {
        List<Token> tokens = new SqlLexer(sql).tokenize();
        SqlStatement statement = new SqlParser(tokens).parseStatement();
        return execute(statement, TransactionContext.AUTOCOMMIT);
    }

    public QueryResult execute(SqlStatement statement, TransactionContext txn) {
        ExecutionVisitor visitor = new ExecutionVisitor(storageEngine, catalog, txn);
        return statement.accept(visitor);
    }
}
```

The ugly part:

```
switch (statement) {
    case SelectStatement ...
    case InsertStatement ...
    ...
}
```

is gone.

```
SqlStatement
|
v
accept (ExecutionVisitor)
|
+----> visit (SelectStatement)
|
+----> visit (InsertStatement)
|
+----> visit (UpdateStatement)
|
+----> ...
```

5. But there's an important trade-off

Your current architecture has a very nice property:

```
SelectStatement -> SelectExecutor
InsertStatement -> InsertExecutor
UpdateStatement -> UpdateExecutor
...
```

Each executor has one responsibility. Visitor instead gives you:

ExecutionVisitor

```
|
+-- Select
+-- Insert
+-- Update
+-- Delete
+-- CreateTable
+-- DropTable
```

So execution logic becomes centralized.

That can get unpleasant as the database grows.

Imagine you later add:

```
ExplainStatement      CreateIndexStatement
BeginStatement        DropIndexStatement
CommitStatement       AlterTableStatement
RollbackStatement
```

Your visitor becomes:

```
public final class ExecutionVisitor implements SqlStatementVisitor<QueryResult> {
    visit(SelectStatement ...)
    visit(InsertStatement ...)
    visit(UpdateStatement ...)
    visit>DeleteStatement ...)
    visit(CreateTableStatement ...)
    visit(DropTableStatement ...)
    visit(ExplainStatement ...)
    visit(BeginStatement ...)
    visit(CommitStatement ...)
    visit(RollbackStatement ...)
    visit(CreateIndexStatement ...)
    visit(DropIndexStatement ...)
    visit(AlterTableStatement ...)
}
```

6. You can combine Visitor with your Executors

This is probably the architecture I'd consider for your example. Keep your six executors, but make the visitor the dispatch mechanism.

```
public final class ExecutionVisitor implements SqlStatementVisitor<QueryResult> {
    private final SelectExecutor selectExecutor;
    private final InsertExecutor insertExecutor;
    private final UpdateExecutor updateExecutor;
    private final DeleteExecutor deleteExecutor;
    private final CreateTableExecutor createTableExecutor;
    private final DropTableExecutor dropTableExecutor;

    public ExecutionVisitor(Catalog catalog, StorageEngine storageEngine) {
        this.selectExecutor = new SelectExecutor(storageEngine);
        this.insertExecutor = new InsertExecutor(storageEngine, catalog);
        this.updateExecutor = new UpdateExecutor(storageEngine);
        this.deleteExecutor = new DeleteExecutor(storageEngine);
        this.createTableExecutor = new CreateTableExecutor(catalog, storageEngine);
        this.dropTableExecutor = new DropTableExecutor(catalog, storageEngine);
    }

    @Override
    public QueryResult visit(SelectStatement statement) {
        return selectExecutor.execute(statement, TransactionContext.AUTOCOMMIT);
    }

    @Override
    public QueryResult visit(InsertStatement statement) {
        return insertExecutor.execute(statement, TransactionContext.AUTOCOMMIT);
    }

    @Override
    public QueryResult visit(UpdateStatement statement) {
        return updateExecutor.execute(statement, TransactionContext.AUTOCOMMIT);
    }

    // ...
}
```

But I'd actually pass the transaction into the visitor rather than hard-code **AUTOCOMMIT**:

```
public final class ExecutionVisitor implements SqlStatementVisitor<QueryResult> {
    private final SelectExecutor selectExecutor;
    private final InsertExecutor insertExecutor;
    // ...

    private final TransactionContext txn;

    public ExecutionVisitor(Catalog catalog, StorageEngine storageEngine, TransactionContext txn) {
        this.selectExecutor = new SelectExecutor(storageEngine);
        this.insertExecutor = new InsertExecutor(storageEngine, catalog);
        // ...

        this.txn = txn;
    }

    @Override
    public QueryResult visit(SelectStatement statement) {
        return selectExecutor.execute(statement, txn);
    }

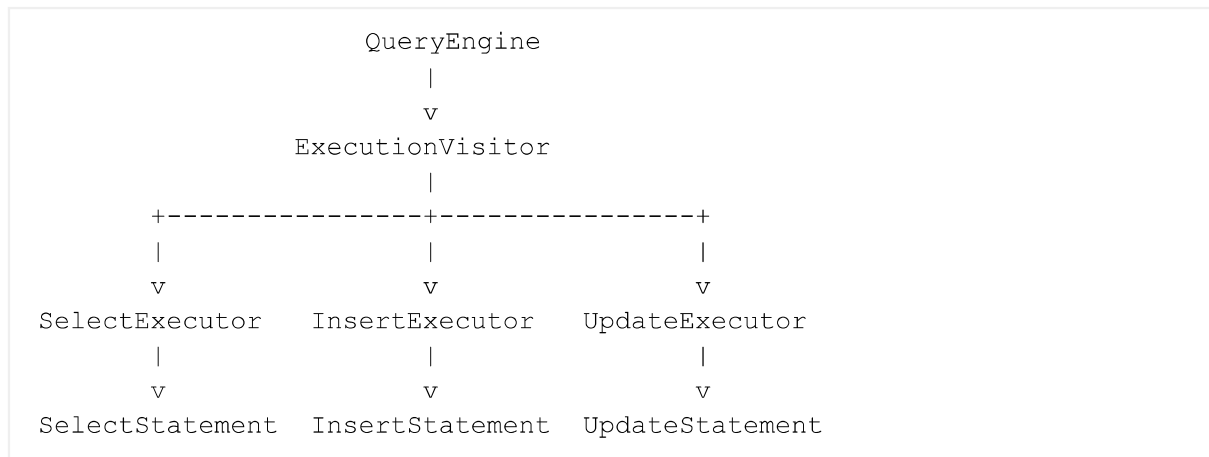
    @Override
    public QueryResult visit(InsertStatement statement) {
        return insertExecutor.execute(statement, txn);
    }

    // ...
}
```

```
public QueryResult execute(
    SqlStatement statement,
    TransactionContext txn) {

    ExecutionVisitor visitor =
        new ExecutionVisitor(
            catalog,
            storageEngine,
            txn);

    return statement.accept(visitor);
}
```



7. Where Visitor becomes especially useful

The really interesting part is that execution isn't the only operation you'll eventually want to perform on your AST.

Suppose you want:

```

SqlStatement
|
+-- execution
+-- validation
+-- EXPLAIN
+-- query optimization
+-- collecting referenced tables
+-- collecting referenced columns
+-- authorization checking
  
```

Without Visitor, you might end up repeatedly doing:

```

if (statement instanceof SelectStatement) {
    ...
} else if (statement instanceof InsertStatement) {
    ...
}
  
```

or repeatedly writing switches. With Visitor, each operation gets its own visitor:

```

ExecutionVisitor
ValidationVisitor
ExplainVisitor
TableReferenceVisitor
ColumnReferenceVisitor
OptimizationVisitor
  
```

```

public final class TableReferenceVisitor
    implements SqlStatementVisitor<Set<String>> {

    @Override
    public Set<String> visit(SelectStatement statement) {
        return Set.of(statement.table());
    }

    @Override
    public Set<String> visit(InsertStatement statement) {
        return Set.of(statement.table());
    }

    @Override
    public Set<String> visit(UpdateStatement statement) {
        return Set.of(statement.table());
    }

    // ...
}
  
```

Then:

```

Set<String> tables = statement.accept(new TableReferenceVisitor());
  
```

Phase 8 — Query Planning: Turning the AST into an Execution Plan

8.1. Why Do You Need a Planner?

Your current executor essentially does this:

```
storageEngine.scan(table)
  .stream()
  .filter(...)
  .map(...)
  .toList();
```

That works.

The problem is that the executor is now deciding how the query should execute.

For example:

```
SELECT name
FROM users
WHERE age >= 18;
```

Our executor decides:

```
scan users
↓
filter age >= 18
↓
project name
```

That is okay for a tiny database.

But consider:

```
SELECT u.name
FROM users u
JOIN orders o
  ON u.id = o.user_id
WHERE u.age >= 18
ORDER BY o.total DESC
LIMIT 10;
```

There could be many possible execution strategies:

There could be many possible execution strategies:

Option A

```
Scan users
↓
Filter age >= 18
↓
Scan orders
↓
Nested-loop join
↓
Sort
↓
Limit
```

Option B

```
IndexScan users(age >= 18)
↓
Scan orders
↓
HashJoin
↓
Sort
↓
Limit
```

Option C

```
Scan orders
↓
Join users
↓
Filter
↓
Sort
↓
Limit
```

The executor should not be responsible for deciding among these. That is the planner/optimizer's job.

So the fundamental reason for a planner is:

Separate "what the query means" from "how the database will execute it."

Without a Planner:

```
SQL
↓
Lexer
↓
Parser
↓
AST
↓
Executor
├── StorageEngine
├── Catalog
└── TransactionContext
```

With a Planner

```
SQL
↓
Lexer
↓
Parser
↓
AST
↓
Semantic Analyzer
↓
Logical Planner
↓
Logical Plan
```

```
↓
Optimizer
↓
Optimized Logical Plan
↓
Physical Planner
↓
Physical Plan
↓
Executor
↓
Storage Engine
```

8.2. Planner vs Optimizer

They are not the same thing.

Planner

The planner answers:

"What relational operations represent this SQL query?"

Example:

```
SELECT name
FROM users
WHERE age >= 18;
```

becomes:

```
Project(name)
|
Filter(age >= 18)
|
Scan(users)
```

The planner is primarily a translation step.

Optimizer

The optimizer answers:

"Can I execute this plan more efficiently without changing its result?"

For Example:

```
Project(name)
|
Filter(age >= 18)
|
Scan(users)
```

might becomes:

```
Project(name)
|
Filter(age >= 18)
|
IndexScan(users, age)
```

Or:

```
Project(...)
|
Filter(...)
|
Join(...)
```

could have predicates pushed closer to the tables.

8.3. Logical Planner vs Physical Planner

Logical Planner:

Transform:

Validated AST → LogicalPlan

Example:

```
SELECT name
FROM users
WHERE age >= 18;
```

becomes:

```
LogicalProject(name)
|
LogicalFilter(age >= 18)
|
LogicalScan(users)
```

The logical plan doesn't care whether the table is stored:

- in memory / on disk
- in pages / behind an index
- in a B+ tree ...

Physical Planner

Transform: **Optimized LogicalPlan → PhysicalPlan**

For Example:

```
LogicalScan(users)
```

Could becomes:

```
TableScan(users)
Or
IndexScan(users_age_index)
```

Logical Plan public sealed interface LogicalPlan permits LogicalScan, LogicalFilter, LogicalProject, LogicalJoin, LogicalAggregate, LogicalSort, LogicalLimit { } 	Then: public record LogicalScan (TableMetadata table) implements LogicalPlan {} public record LogicalFilter (LogicalPlan input, Expression predicate) implements LogicalPlan {} public record LogicalProject (LogicalPlan input, List<Expression> expressions) implements LogicalPlan {}
---	--

The prompt defines this logical-plan model and extends it with joins, aggregation, sorting and limits.

8.4. Step-by-Step Query Planner

For a SELECT, your planner should conceptually perform these steps.

Step 1 — Start with FROM FROM users becomes LogicalScan(users) 	Step 2 — Add JOIN FROM users u JOIN orders o ON u.id = o.user_id becomes <pre> Join(u.id = o.user_id) / \ Scan(users) Scan/orders) </pre> 	Step 3 — Add WHERE WHERE age >= 18 becomes <pre> Filter(age >= 18) Join </pre>
Step 4 — Add GROUP BY GROUP BY department becomes Aggregate(group = department) 	Step 5 — Add HAVING HAVING COUNT(*) > 5 becomes becomes a filter over the aggregate result. 	Step 6 — Add SELECT SELECT name, age becomes Project(name, age)
Step 7 — Add ORDER BY ORDER BY age DESC becomes Sort(age DESC) 	Step 5 — Add LIMIT LIMIT 10 becomes Limit(10) 	

8.5. Optimizer

```
public interface OptimizationRule {  
    boolean matches(LogicalPlan plan);  
    LogicalPlan apply(LogicalPlan plan);  
}
```

```
public final class PlanOptimizer {  
    private final List<OptimizationRule> rules;  
  
    public LogicalPlan optimize(LogicalPlan plan) {  
        LogicalPlan current = plan;  
        boolean changed;  
  
        do {  
            changed = false;  
            for (OptimizationRule rule : rules) {  
                if (rule.matches(current)) {  
                    LogicalPlan next = rule.apply(current);  
                    if (next != current) {  
                        current = next;  
                        changed = true;  
                    }  
                }  
            }  
        } while (changed);  
        return current;  
    }  
}
```

Example Optimization:

Suppose the planner generates:

```
Project(name)  
  |  
Filter(age > 18)  
  |  
Scan(users)
```

An optimizer may eventually push the filter closer to the scan.:

For more complex joins:	might become:
<pre>Project Join / \ Scan Filter</pre>	<pre>Project Join / \ Filter Scan Scan</pre>

This is called **predicate pushdown**.

8.6. The Plan Interface

```
public interface Plan {  
    Scan open(); // §29 -- calling this does not read any data yet, only prepares to  
    int blocksAccessed(); // cost estimate, for a future CBO to compare across candidate plans  
    int recordsOutput();  
    TableSchema schema();  
}
```

8.6.1. TablePlan — the only **Plan** that talks to **StorageEngine** directly:

```
public final class TablePlan implements Plan {  
    private final String tableName;  
    private final StorageEngine storageEngine;  
    private final Catalog catalog;  
  
    @Override  
    public Scan open() { return storageEngine.openScan(tableName); }  
  
    @Override  
    public int blocksAccessed() { return catalog.getStatistics(tableName).blockCount(); }  
  
    @Override  
    public int recordsOutput() { return catalog.getStatistics(tableName).recordCount(); }  
  
    @Override  
    public TableSchema schema() { return catalog.getTable(tableName); }  
}
```

8.6.2. **SelectPlan** and **ProjectPlan** — each wraps another **Plan**, the way a **Decorator** wraps another object, rather than talking to storage itself:

```
public final class SelectPlan implements Plan {
    private final Plan input;
    private final Expression predicate;

    @Override
    public Scan open() { return new SelectScan(input.open(), predicate); } // §30

    @Override // // filtering reads the SAME blocks -- no extra I/O
    public int blocksAccessed() { return input.blocksAccessed(); }

    @Override // // a naive fixed selectivity -- a CBO refines this
    public int recordsOutput() { return input.recordsOutput() / 2; }

    @Override // // filtering ROWS never changes which COLUMNS exist
    public TableSchema schema() { return input.schema(); }
}
```

8.6.3. **ProjectPlan**:

```
public final class ProjectPlan implements Plan {
    private final Plan input;
    private final List<String> fields;

    @Override // // §30
    public Scan open() { return new ProjectScan(input.open(), fields); }

    @Override
    public int blocksAccessed() { return input.blocksAccessed(); }

    @Override // // projecting columns doesn't change row COUNT
    public int recordsOutput() { return input.recordsOutput(); }

    @Override // // a NARROWER schema -- fewer columns
    public TableSchema schema() { return input.schema().project(fields); }
}
```

TablePlan, **SelectPlan**, and **ProjectPlan** compose exactly the way **SelectStatement's** three parts (table, where, columns) already suggest: read, then filter, then narrow columns — each step wrapping the previous **Plan**, never reaching around it to touch storage directly.

8.7. BasicQueryPlanner and UpdatePlanner

```
public interface QueryPlanner {
    Plan createPlan(SelectStatement statement, TransactionContext tx);
}
```

```
public interface UpdatePlanner {
    int executeInsert(InsertStatement statement, TransactionContext tx);
    int executeUpdate(UpdateStatement statement, TransactionContext tx);
    int executeDelete(DeleteStatement statement, TransactionContext tx);
    int executeCreateTable(CreateTableStatement statement, TransactionContext tx);
    int executeDropTable(DropTableStatement statement, TransactionContext tx);
}
```

```
public final class BasicQueryPlanner implements QueryPlanner {
    private final Catalog catalog;
    private final StorageEngine storageEngine;

    @Override
    public Plan createPlan(SelectStatement statement, TransactionContext tx) {
        Plan plan = new TablePlan(statement.table(), storageEngine, catalog); // Step 1: read
        if (statement.where() != null) plan = new SelectPlan(plan, statement.where()); // Step 2: filter rows
        boolean selectStar = statement.columns().size() == 1 && statement.columns().get(0).equals("**");
        if (!selectStar) plan = new ProjectPlan(plan, statement.columns()); // Step 3: filter columns
        return plan;
    }
}
```

```
public final class BasicUpdatePlanner implements UpdatePlanner {
    private final Catalog catalog; // constructor injection
    private final StorageEngine storageEngine; // constructor injection

    public int executeInsert(InsertStatement statement, TransactionContext tx) {
        Plan plan = new TablePlan(statement.table(), storageEngine, catalog);
        Scan scan = plan.open();
        scan.insert(); // position at a fresh slot
        for (int i = 0; i < statement.columns().size(); i++) {
            scan.setVal(statement.columns().get(i), statement.values().get(i));
        }
        scan.close();
        return 1;
    }

    public int executeUpdate(UpdateStatement statement, TransactionContext tx) {
        Plan plan = new TablePlan(statement.table(), storageEngine, catalog);
        if (statement.where() != null) plan = new SelectPlan(plan, statement.where());
        Scan scan = plan.open();
        int affected = 0;
        while (scan.next()) {
            for (var entry : statement.assignments().entrySet()) scan.setVal(entry.getKey(), entry.getValue());
            affected++;
        }
        scan.close();
        return affected;
    }

    public int executeDelete(DeleteStatement statement, TransactionContext tx) {
        Plan plan = new TablePlan(statement.table(), storageEngine, catalog);
        if (statement.where() != null) plan = new SelectPlan(plan, statement.where());
        Scan scan = plan.open();
        int affected = 0;
        while (scan.next()) { scan.delete(); affected++; }
        scan.close();
        return affected;
    }
}
```

```

public int executeCreateTable(CreateTableStatement statement, TransactionContext tx) {
    catalog.createTable(statement.schema());
    storageEngine.createTable(statement.schema());
    return 0;
}

public int executeDropTable(DropTableStatement statement, TransactionContext tx) {
    catalog.dropTable(statement.table());
    storageEngine.dropTable(statement.table());
    return 0;
}
}

```

8.8. BasicPlanner: A Facade Over Parsing and Planning

BasicPlanner is a **Facade** sitting between the parser and the two planners above — the same seam idea as §18's **QueryEngine**, one layer deeper now that statements produce **Plans** instead of being executed immediately:

```

public final class BasicPlanner {
    private final QueryPlanner queryPlanner;
    private final UpdatePlanner updatePlanner;

    public Plan createQueryPlan(String sql, TransactionContext tx) {
        SqlStatement statement = parse(sql);
        if (!(statement instanceof SelectStatement select))
            throw new IllegalArgumentException("Not a query: " + sql);
        return queryPlanner.createPlan(select, tx);
    }

    public int executeUpdate(String sql, TransactionContext tx) {
        SqlStatement statement = parse(sql);
        return switch (statement) {
            case InsertStatement s -> updatePlanner.executeInsert(s, tx);
            case UpdateStatement s -> updatePlanner.executeUpdate(s, tx);
            case DeleteStatement s -> updatePlanner.executeDelete(s, tx);
            case CreateTableStatement s -> updatePlanner.executeCreateTable(s, tx);
            case DropTableStatement s -> updatePlanner.executeDropTable(s, tx);
            case SelectStatement s -> throw new IllegalArgumentException("Use createQueryPlan for a
SELECT");
        };
    }

    private SqlStatement parse(String sql) {
        List<Token> tokens = new SqlLexer(sql).tokenize(); // §7
        return new SqlParser(tokens).parseStatement(); // §11
    }
}

```

Phase 9 — Query Execution: Producing Results

Why Direct Execution Doesn't Scale: Introducing the Planner

§17's executors work correctly, and they share one real limitation: **SelectExecutor.execute** always calls **storageEngine.scan(table)** — a full table scan, every single time, even for **WHERE id = 100** against a table with a primary-key index that could answer the same question in **O(log n)** instead of **O(n)**. Nothing in §17's code has anywhere to record "use the index here" as a decision — the scan-then-filter logic is hard-coded inline. Fixing this requires separating "what to do" (a **Plan**, an object you can inspect, compare, and choose between) from "just doing it" — which is the entire reason a query planner exists.

The Three Parts of a Query Engine

A query engine is not one component. It is three, each with a distinct job:

Query Optimizer	-- decides HOW to answer a query (which plan, which indexes)
Execution Engine	-- actually reads rows/columns according to that plan
Catalog / Metadata	-- knows what tables and indexes exist, and their shape

Part 4 (this part) builds the Optimizer's mechanism and the seam it plugs into. Part 5 builds the Execution Engine. Part 6 builds the Catalog Manager for real. Keeping these three separate is the same Interface Segregation discipline (a **QueryPlanner**, a **QueryExecutor**, and a **Catalog** as three different interfaces) that motivates never merging them into one **DatabaseEverything** class.

Rule-Based vs Cost-Based Optimization

"Use the index if the **WHERE** clause matches an indexed column, otherwise scan the table" is **Rule-Based Optimization (RBO)** — a fixed rule, applied without comparing alternatives. A **Cost-Based Optimizer (CBO)** generalizes this: instead of one hard-coded rule, it generates several candidate plans and picks whichever has the lowest estimated cost, using statistics the Catalog Manager (Part 6) collects — row counts, blocks accessed, selectivity. RBO is simpler to build and reason about; CBO scales better as the number of indexes and join orderings grows, at the cost of needing real statistics to estimate against. This guide builds RBO's actual mechanism — §27's **Plan.blocksAccessed()/recordsOutput()** are exactly the hooks a later CBO would compare across candidate plans.

9.1. BasicQueryEngine: doQuery and doUpdate

The externally-visible database facade — what a CLI or a JDBC driver actually calls:

```
public final class BasicQueryEngine {
    private final BasicPlanner planner;
    private final TransactionManager txnManager;

    public QueryResult doQuery(String sql) {
        TransactionContext tx = txnManager.begin();
        Plan plan = planner.createQueryPlan(sql, tx);
        Scan scan = plan.open();

        List<Row> rows = new ArrayList<>();
        while (scan.next()) {
            Map<String, Object> values = new LinkedHashMap<>();
            for (String field : plan.schema().fieldNames()) values.put(field, scan.getVal(field));
            rows.add(new Row(values));
        }
        scan.close();
        txnManager.commit(tx);
        return QueryResult.ofRows(rows);
    }

    public QueryResult doUpdate(String sql) {
        TransactionContext tx = txnManager.begin();
        int affected = planner.executeUpdate(sql, tx);
        transactionManager.commit(tx);
        return QueryResult.ofAffected(affected);
    }
}
```

doQuery is the only place left that walks a **Scan** into a **List<Row>** — and it does that only because the caller above it wants a materialized result set to print or return. The engine itself, all the way down through **Plan/Scan**, never materializes more than one row at a time (§32 explains exactly why that matters).

9.2. Revisiting the Six Executors to Use the Planner

```
// SelectExecutor, REVISITED: same StatementExecutor<SelectStatement> contract as §17,  
// but the BODY now delegates to the planner instead of scanning+filtering a  
// materialized List<Row> by hand.  
public final class SelectExecutor implements StatementExecutor<SelectStatement> {  
    private final QueryPlanner queryPlanner;  
  
    @Override  
    public QueryResult execute(SelectStatement statement, TransactionContext txn) {  
        Plan plan = queryPlanner.createPlan(statement, txn);  
        Scan scan = plan.open();  
  
        List<Row> rows = new ArrayList<>();  
        while (scan.next()) {  
            Map<String, Object> values = new LinkedHashMap<>();  
            for (String field : plan.schema().fieldNames()) values.put(field, scan.getVal(field));  
            rows.add(new Row(values));  
        }  
        scan.close();  
        return QueryResult.ofRows(rows);  
    }  
}
```

The public contract did not change at all. Only what happens inside **execute** changed — from "load everything, then filter in Java" to "compose a plan, then stream through it." This is precisely why §17 was told to depend on **Catalog/StorageEngine** interfaces: the same Dependency Inversion that let **MemoryStorageEngine** be swapped for a real one also lets an executor's implementation strategy change without its callers noticing.

The Scan Interface: Streaming Instead of Materializing

`StorageEngine.scan(tableName)` returns a `List<Row>` — every row, loaded into memory, before anything even looks at the first one. That is fine for a table that fits comfortably in memory; it is a real problem the moment a table doesn't. A Scan fixes this by being an iterator, not a collection — it produces rows one at a time, on demand:

```
public interface Scan {  
    boolean next();  
    Object getVal(String fieldName);  
    boolean hasField(String fieldName);  
    void close();  
  
    // Writable operations -- a stricter design (see the note below) splits these  
    // into a separate interface so a read-only SELECT path can never call them.  
    void insert();  
    void setVal(String fieldName, Object value);  
    void delete();  
}
```

Putting both read and write methods on one interface is a pragmatic simplification, not the final answer — Interface Segregation argues against exactly this shape in the abstract. A stricter design splits this into **ReadOnlyScan** (`next/getVal/hasField/close`) and **UpdatableScan** extends **ReadOnlyScan** (adding `insert/setVal/delete`), so a **SELECT** path is never even able to call `delete()` by accident — the compiler enforces it instead of a runtime check. §29's implementations work identically either way; only which interface each one is declared to return changes.

TableScan

TableScan is the only Scan that talks to **StorageEngine** directly — everything else wraps another Scan:

```
// StorageEngine (§16) gains one method beyond insert()/scan()/replaceRows():
public interface StorageEngine {
    // ... createTable, dropTable, insert, scan, replaceRows from §16 ...
    Scan openScan(String tableName);
}
```

```
// MemoryStorageEngine.openScan -- still backed by the in-memory List<Row> for now.
// Once real page-based storage exists (the companion Storage Engine guide), this walks
// Pages via RowId instead of a Java Iterator -- the Scan CONTRACT above does not change.
@Override
public Scan openScan(String tableName) {
    List<Row> rows = tables.get(tableName);
    return new Scan() {
        private int index = -1;

        @Override
        public boolean next() {
            index++;
            return index < rows.size();
        }

        @Override
        public Object getVal(String fieldName) { return rows.get(index).get(fieldName); }

        @Override
        public boolean hasField(String fieldName) {
            return rows.get(index).columnNames().contains(fieldName);
        }

        @Override
        public void insert() {
            rows.add(new Row(new HashMap<>()));
            index = rows.size() - 1;
        }

        @Override
        public void setVal(String fieldName, Object value) {
            Map<String, Object> updated = new LinkedHashMap<>();
            for (String column : rows.get(index).columnNames())
                updated.put(column, rows.get(index).get(column));
            updated.put(fieldName, value);
            rows.set(index, new Row(updated));
        }

        @Override
        public void delete() {
            rows.remove(index);
            index--; // the next next() call must not skip the row that just slid into this slot
        }

        @Override
        public void close() {}
    };
}
```

SelectScan and ProjectScan

SelectScan wraps a child **Scan** and a predicate — it filters rows as they stream past, one at a time, never holding more than the current row:

```
public final class SelectScan implements Scan {
    private final Scan input;
    private final Expression predicate;

    public SelectScan(Scan input, Expression predicate) {
        this.input = input;
        this.predicate = predicate;
    }

    @Override
    public boolean next() {
        while (input.next()) {
            if (ExpressionEvaluator.matchesScan(this, predicate)) return true; // §31
        }
        return false;
    }

    @Override public Object getVal(String fieldName) { return input.getVal(fieldName); }
    @Override public boolean hasField(String fieldName) { return input.hasField(fieldName); }
    @Override public void setVal(String fieldName, Object value) { input.setVal(fieldName, value); }
    @Override public void insert() { input.insert(); }
    @Override public void delete() { input.delete(); }
    @Override public void close() { input.close(); }
}
```

```
public final class ProjectScan implements Scan {
    private final Scan input;
    private final List<String> fields;

    @Override
    public boolean next() { return input.next(); }

    @Override
    public Object getVal(String fieldName) {
        if (!hasField(fieldName)) throw new RuntimeException("field " + fieldName + " not found.");
        return input.getVal(fieldName);
    }

    @Override
    public boolean hasField(String fieldName) { return fields.contains(fieldName); }

    @Override public void setVal(String fieldName, Object value) { throw new
        UnsupportedOperationException("read-only projection"); }
    @Override public void insert() { throw new UnsupportedOperationException("read-only
        projection"); }
    @Override public void delete() { throw new UnsupportedOperationException("read-only
        projection"); }
    @Override public void close() { input.close(); }
}
```

ProjectScan throwing on every write method is worth noticing — projection is fundamentally a read-shaping operation, and §28's "split into **ReadOnlyScan/UpdatableScan**" refinement would let this class simply not implement the write methods at all, rather than implementing them just to reject every call.

Extending the Expression Evaluator to Work Against a Scan

SelectScan (§30) needs to evaluate the same **Expression** tree §14 already built — but against a **Scan**'s current position, not a fully-formed **Row**. Extend **ExpressionEvaluator** with a second entry point that mirrors **matches(Row, Expression)** field for field, reading through **Scan.getVal(...)** instead of **Row.get(...)**:

```
// Added to ExpressionEvaluator (§14):
public static boolean matchesScan(Scan scan, Expression expression) {
    return switch (expression) {
        case Comparison c -> evaluateComparison(scan.getVal(c.column()), c);
        case And a -> matchesScan(scan, a.left()) && matchesScan(scan, a.right());
        case Or o -> matchesScan(scan, o.left()) || matchesScan(scan, o.right());
    };
}
```

§14's **evaluateComparison** was already written to take the actual value directly (not a **Row**), specifically so both **matches(Row, ...)** and **matchesScan(Scan, ...)** can share that one comparison method instead of duplicating it — one comparison implementation, two entry points, which is what keeps **Row**-based execution (Part 3) and **Scan**-based execution (Part 5) from silently drifting into two different definitions of "matches."

32. Why Streaming Beats Materializing: A Concrete Cost Trace

Trace what happens for: "SELECT name FROM employee WHERE salary > 50000;" against a table of one million rows where ten thousand match:

```
Part 3's original SelectExecutor (§17):
storageEngine.scan("employee") → loads ALL 1,000,000 rows into a List<Row>
.stream().filter(...)           → then filters down to 10,000, still in memory
.map(...)                       → then projects, producing a THIRD list

Part 4-5's Plan/Scan pipeline (§27-30):
TableScan.next() -> SelectScan.next() → ProjectScan.next()
-- pulls ONE row at a time through all three stages, filters it, projects it, and only
the caller (BasicQueryEngine.doQuery, §26) decides whether to keep it in a List --
the pipeline itself never holds more than one row per stage at once.
```

Both eventually produce the same 10,000-row result. The difference is **peak memory**: one materializes three full intermediate collections along the way; the other never holds more than a handful of rows at any given instant, regardless of whether the table has a thousand rows or a billion. This is the concrete, measurable payoff of the Iterator-shaped **Scan** interface (§28) over §17's original "load a list, then **.stream()** it" approach — and it is also exactly why **Sort** and **Aggregate** (Part 9's future work) are structurally different from **Filter/Project**: sorting the whole result requires seeing every row before producing the first one, so a **SortScan** cannot stream the way **SelectScan/ProjectScan** do — a real, unavoidable limit on how far the streaming idea extends, worth stating honestly rather than implying every operator can be lazy for free.

Phase 10 — The Catalog Interface, Revisited

§9's **Catalog** interface has been used throughout this guide without a real, durable implementation — every example so far implicitly assumes something like a **Map<String, TableSchema>** in memory, which forgets every table the instant the process restarts. Part 6 fixes that properly.

```
public interface Catalog {
    void createTable(TableSchema schema);
    void dropTable(String tableName);
    TableSchema getTable(String tableName);
    boolean tableExists(String tableName);
    List<TableSchema> listTables();
    TableStatistics getStatistics(String tableName); // consumed by
    TablePlan.blocksAccessed()/recordsOutput(), §23
}

public record TableStatistics(int blockCount, int recordCount) { }
```

Phase 11 — TablePhysicalLayout: From Logical Schema to Byte Offsets

TableSchema (§9) is a table's logical shape — names, types, nullability. Computing a row's exact physical layout needs one more thing: the byte offset of each column within a row, and the row's total **slot size**:

```
public final class TablePhysicalLayout {
    private final Map<String, Integer> offsets = new HashMap<>();
    private final int slotSize;

    public TablePhysicalLayout(TableSchema schema) {
        int position = 0;
        for (Column column : schema.columns()) {
            offsets.put(column.name(), position);
            position += sizeInBytes(column);
        }
        this.slotSize = position;
    }

    private int sizeInBytes(Column column) {
        return switch (column.type()) {
            case INT -> 4;
            case BIGINT -> 8;
            case BOOLEAN -> 1;
            case VARCHAR -> 50; // fixed-length slots for simplicity -- even VARCHAR gets a fixed
            budget
            case DECIMAL -> 12;
            case DATE, TIMESTAMP -> 8;
        };
    }

    public int offset(String fieldName) { return offsets.get(fieldName); }
    public int slotSize() { return slotSize; }
}
```

Fixed-length slots (even for **VARCHAR**) is a deliberate simplifying decision — worth restating because it's precisely what makes **offset(fieldName)** a single, cheap arithmetic lookup instead of "scan every earlier column to find where this one starts." The companion **Storage Engine guide** uses exactly this layout to encode/decode rows at the byte level.

Phase 12 — PersistentCatalog: Storing Schema as Data

A real database stores its own schema as ordinary rows, in special tables the database itself manages, so table definitions get the exact same durability (WAL, covered in the companion Storage Engine guide) as user data, instead of a separate, ad hoc persistence mechanism:

tinydb_tables			tinydb_columns			
tblname	slotsize		tblname	fldname	type	length
employee	48		employee	id	BIGINT	0
			employee	name	VARCHAR	50

```
public final class PersistentCatalog implements Catalog {
    private static final TableSchema TABLES_SCHEMA = new TableSchema("tinydb_tables", List.of(
        new Column("tblname", DataType.VARCHAR, false), new Column("slotsize", DataType.INT, false))
    );

    private static final TableSchema COLUMNS_SCHEMA = new TableSchema("tinydb_columns", List.of(
        new Column("tblname", DataType.VARCHAR, false), new Column("fldname", DataType.VARCHAR,
false),
        new Column("type", DataType.VARCHAR, false), new Column("length", DataType.INT, false))
    );

    private final StorageEngine storageEngine;

    public PersistentCatalog(StorageEngine storageEngine) {
        this.storageEngine = storageEngine;
        storageEngine.createTable(TABLES_SCHEMA); // bootstrap: the catalog's own tables must exist first
        storageEngine.createTable(COLUMNS_SCHEMA);
    }

    @Override
    public void createTable(TableSchema schema) {
        int slotSize = new TablePhysicalLayout(schema).slotSize(); // §34
        storageEngine.insert(TABLES_SCHEMA.tableName(),
            new Row(Map.of("tblname", schema.tableName(), "slotsize", slotSize)));
        for (Column column : schema.columns()) {
            storageEngine.insert(COLUMNS_SCHEMA.tableName(), new Row(Map.of(
                "tblname", schema.tableName(), "fldname", column.name(),
                "type", column.type().name(), "length", 0)));
        }
    }

    @Override
    public TableSchema getTable(String tableName) {
        boolean exists = storageEngine.scan(TABLES_SCHEMA.tableName()).stream()
            .anyMatch(row -> row.get("tblname").equals(tableName));
        if (!exists) return null;

        List<Column> columns = storageEngine.scan(COLUMNS_SCHEMA.tableName()).stream()
            .filter(row -> row.get("tblname").equals(tableName))
            .map(row -> new Column((String) row.get("fldname"), DataType.valueOf((String) row.get("type")),
true))
            .toList();
        return new TableSchema(tableName, columns);
    }

    // dropTable/tableExists/listTables/getStatistics omitted for brevity -- same
    // "read from tinydb_tables / tinydb_columns" pattern as getTable() above.
}
```

Phase 18 — PersistentCatalog: Storing Schema as Data

A real database stores its own schema as ordinary rows, in special tables the database itself manages, so table definitions get the exact same durability (WAL, covered in the companion Storage Engine guide) as user data, instead of a separate, ad hoc persistence mechanism:

tinydb_tables			tinydb_columns			
tblname	slotsize		tblname	fldname	type	length
employee	48		employee	id	BIGINT	0
			employee	name	VARCHAR	50

```
public final class PersistentCatalog implements Catalog {
    private static final TableSchema TABLES_SCHEMA = new TableSchema("tinydb_tables", List.of(
        new Column("tblname", DataType.VARCHAR, false), new Column("slotsize", DataType.INT, false))
    );

    private static final TableSchema COLUMNS_SCHEMA = new TableSchema("tinydb_columns", List.of(
        new Column("tblname", DataType.VARCHAR, false), new Column("fldname", DataType.VARCHAR,
false),
        new Column("type", DataType.VARCHAR, false), new Column("length", DataType.INT, false))
    );

    private final StorageEngine storageEngine;

    public PersistentCatalog(StorageEngine storageEngine) {
        this.storageEngine = storageEngine;
        storageEngine.createTable(TABLES_SCHEMA); // bootstrap: the catalog's own tables must exist first
        storageEngine.createTable(COLUMNS_SCHEMA);
    }

    @Override
    public void createTable(TableSchema schema) {
        int slotSize = new TablePhysicalLayout(schema).slotSize(); // §34
        storageEngine.insert(TABLES_SCHEMA.tableName(),
            new Row(Map.of("tblname", schema.tableName(), "slotsize", slotSize)));
        for (Column column : schema.columns()) {
            storageEngine.insert(COLUMNS_SCHEMA.tableName(), new Row(Map.of(
                "tblname", schema.tableName(), "fldname", column.name(),
                "type", column.type().name(), "length", 0)));
        }
    }

    @Override
    public TableSchema getTable(String tableName) {
        boolean exists = storageEngine.scan(TABLES_SCHEMA.tableName()).stream()
            .anyMatch(row -> row.get("tblname").equals(tableName));
        if (!exists) return null;

        List<Column> columns = storageEngine.scan(COLUMNS_SCHEMA.tableName()).stream()
            .filter(row -> row.get("tblname").equals(tableName))
            .map(row -> new Column((String) row.get("fldname"), DataType.valueOf((String) row.get("type")),
true))
            .toList();
        return new TableSchema(tableName, columns);
    }

    // dropTable/tableExists/listTables/getStatistics omitted for brevity -- same
    // "read from tinydb_tables / tinydb_columns" pattern as getTable() above.
}
```

Follow-up: How Does the Planner Actually Choose an IndexScan?

§22 named Rule-Based Optimization's canonical example — "use the index if the **WHERE** clause matches an indexed column" — without yet showing the mechanism. With **Plan** (§23) as a real abstraction, the answer is now concrete: **BasicQueryPlanner.createPlan** (§24) needs one more decision point, made before defaulting to **TablePlan**:

```
@Override
public Plan createPlan(SelectStatement statement, TransactionContext tx) {
    Plan plan = selectAccessPath(statement); // NEW: choose TablePlan OR IndexSelectPlan, per §38

    if (statement.where() != null) plan = new SelectPlan(plan, statement.where());
    boolean selectStar = statement.columns().size() == 1 && statement.columns().get(0).equals("");
    if (!selectStar) plan = new ProjectPlan(plan, statement.columns());
    return plan;
}

private Plan selectAccessPath(SelectStatement statement) {
    if (statement.where() instanceof Comparison c
        && c.operator().equals("=")
        && catalog.hasIndex(statement.table(), c.column())) {
        return new IndexSelectPlan(statement.table(), c.column(), c.value(), catalog); // §38
    }
    return new TablePlan(statement.table(), storageEngine, catalog); // the RBO "default" rule
}
```

catalog.hasIndex(table, column) is the one new **Catalog** method this decision needs — a Cost-Based Optimizer would replace this single **if** with a comparison of **blocksAccessed()** across every viable **Plan**, but the RBO version above is a real, working decision, not a placeholder.

Sketch: An IndexSelectPlan

```
public final class IndexSelectPlan implements Plan {
    private final String tableName;
    private final String indexedColumn;
    private final Object searchKey;
    private final Catalog catalog;

    @Override
    public Scan open() {
        RowId rowId = catalog.lookupIndex(tableName, indexedColumn, searchKey); // O(log n), not O(n)
        return new SingleRowScan(tableName, rowId); // a Scan over exactly one row, or zero if not found
    }

    @Override // a B+Tree lookup touches a small, roughly-constant number of pages
    public int blocksAccessed() { return 3; }

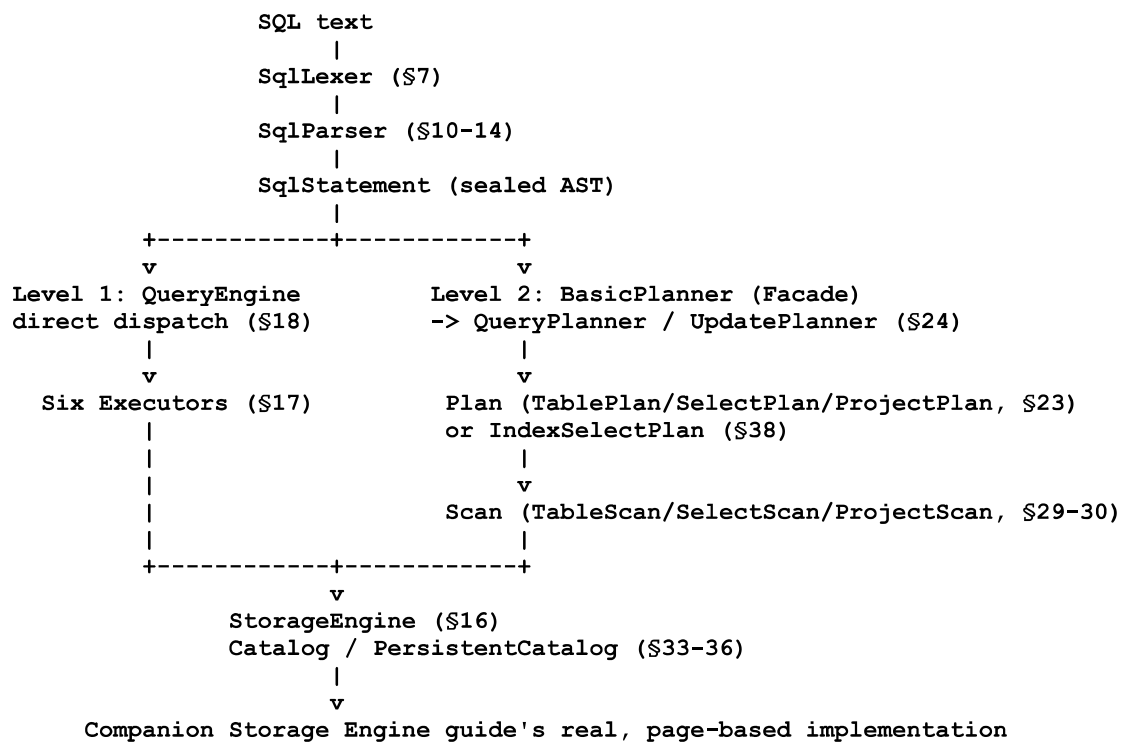
    @Override // an equality lookup on a unique index returns at most one row
    public int recordsOutput() { return 1; }

    @Override
    public TableSchema schema() { return catalog.getTable(tableName); }
}
```

Full Worked Example

```
"SELECT name FROM employee WHERE salary > 50000"
-> SqlLexer.tokenize() (S7)
-> SqlParser.parseStatement() -> SelectStatement (S10-11, S13)
-> BasicQueryEngine.doQuery(sql) (S26)
    -> BasicPlanner.createQueryPlan(sql, tx) (S25)
        -> BasicQueryPlanner.createPlan(statement, tx) (S24, extended by
$37)
            -> selectAccessPath -> TablePlan (no index on "salary")
            -> wrapped in SelectPlan(where: salary > 50000)
            -> wrapped in ProjectPlan(columns: ["name"])
        -> plan.open() -> ProjectScan(SelectScan(TableScan)) (S29-30)
        -> while (scan.next()) { ... } (S26)
-> QueryResult.ofRows([{"name": "Alice"}])
```

Final Architecture



Design Patterns Used

Pattern	Applied to
Command	Each <code>SqlStatement</code> is a self-contained, immutable description of an action
Facade	<code>BasicPlanner/BasicQueryEngine</code> hiding the parser, planner, and executor layers behind <code>doQuery/doUpdate</code>
Decorator	<code>SelectPlan/ProjectPlan</code> and <code>SelectScan/ProjectScan</code> each wrap another <code>Plan/Scan</code>
Iterator	<code>Scan</code> streams rows one at a time, exactly like <code>java.util.Iterator</code>
Strategy	Choosing <code>TablePlan</code> vs <code>IndexSelectPlan</code> per query, without the caller knowing which was chosen
Visitor	ANTLR's generated parse-tree traversal

SOLID Principles Applied

Principle	Where it holds
Single Responsibility	<code>SqlLexer</code> only tokenizes; <code>SqlParser</code> only parses; each executor only executes one statement type
Open/Closed	Adding a seventh <code>SqlStatement</code> type is additive everywhere the sealed interface is switched over — the compiler enforces it (§18, §25)
Liskov Substitution	Any <code>Plan/Scan</code> implementation is fully interchangeable from its caller's point of view
Interface Segregation	<code>QueryPlanner/UpdatePlanner/Catalog</code> are separate, narrow interfaces; §28 names the further split <code>Scan</code> itself deserves
Dependency Inversion	Every executor and planner depends on <code>Catalog/StorageEngine</code> interfaces, never a concrete implementation

Suggested Future Enhancements

Enhancement	Where it plugs in
JOIN support	A <code>JoinPlan/NestedLoopJoinScan</code> pair, composed the same way <code>SelectPlan/SelectScan</code> are
ORDER BY	A <code>SortPlan/SortScan</code> — the one operator §32 already flags as unable to stream
GROUP BY / aggregate functions	An <code>AggregatePlan</code> consuming a full (possibly sorted) input before producing output rows
Real B+Tree-backed indexes	See the companion <code>Storage Engine</code> guide's indexing sections
A real Cost-Based Optimizer	Extending §37's <code>selectAccessPath</code> to build several plans and pick the cheapest via <code>blocksAccessed()</code>
Prepared statements	Parameterizing <code>Comparison.value()</code> instead of baking a literal into the AST at parse time